

3D 게임프로그래밍2 과제 #01

2019184016 서정원

1. 과제에 대한 목표

- ➔ 나는 이번 과제를 통해 3D 게임프로그래밍1 에서 특정 과제를 다시 분석하는 것부터 시작했다. 방학부터 다시 기초로 돌아가 코드에 기재된 모든 항목을 숙지한 상태로 시작하는 목표가 있었습니다. 결과적으로 교수님께서 주신 코드를 병합하는 것도 목표였지만, 코드를 그대로 사용하는 것도 아닌 다른 프레임워크로 구조를 다르게 하여 변수 및 함수 사용을 다르게 하여 구조에 대해 익히는 목표가 있었다. 게임 구현에 대한 목표는 탱크를 로드 하여 텍스처를 입히고 수업에서 배운 노멀매핑도 적용하고 최대한 배운 모든 것을 적용하려고 노력했다. 보고서는 다 마치고 쓰는 것이 아닌 하나하나 구현할 때마다 수정했다.
- ➔ 과제 게임 구현 목표: 일단 필수 구현 요소인 지형 , 스카이박스, 물, 건물, 나무 , 탱크, 이펙트 처리, 점수 표현 등 모두 구현하였다. 지형은 블렌딩을 연습하기 위해 다중 텍스처로 알파 텍스처까지 활용해 알파 값이 있는 부분은 디테일 텍스처까지 입히는 연습을 하였고, 스카이 박스는 직접 제작하면서 맵의 배경에 맞게 괜찮은 사진들을 조합하여 큐브텍스처를 만들었고, 물은 RippleWater를 적용시켜 내부 셰이더에서 cos ,sin함수를 나의 방식으로 조작하여 더 역동적으로 움직이게 함과 동시에 알파 블렌딩을 넣어 물을 투명하게 하였고, 건물은 모델을 불러와 노멀 매핑 등 매핑을 수행하고 충돌처리를 하였고, 나무는 마찬가지로 블렌딩을 넣고 알파커버리지를 넣어 자연스러운 입사귀 효과를 나타냈고, 탱크 등 UI, 사운드, 등등 하나하나 공부하여 직접 텍스처도 다 만들고 과제가 나오기 전부터 연습하던 프로젝트를 이어서 그대로 진행하였다. 진짜 많은 시간을 투자하여 1학기 때 못 따라갔던 부분들을 매우 만족스럽게 공부했다고 생각할 만큼 복습하고 연습하고 해서 만족스러운 결과가 나왔다고 생각한다.
- ➔ 게임 진행 : 게임은 처음 실행 시 시작 화면이 뜬다. 시작 썬에서 Space 키의 입력을 기다려 스페이스 키를 입력하면 게임 썬으로 넘어간다. 주어진 시간은 3분이며, 3분동안 적 탱크 객체를 7개를 모두 죽여야 게임에서 승리를 할 수 있다. 만약 3분이 경과된다면 게임에서 패배를 하게 된다. 게임을 시작하면 10초 간격으로 지형과 물의 색이 진흙 색으로 바뀐다. 이 때 플레이어는 이동속도가 느려진다. 만약 지형이 진흙 색이 아니라면 Accelerate를 할 수 있고 만약 그냥 움직

이면 기본 이동속도로 움직인다. 적 객체는 건물과 돌, 등에 충돌은 하지 않지만 플레이어는 충돌하기 때문에 우회해서 찾아 쏘야한다. 모든 탱크는 미사일을 한번 맞으면 죽는다. 각 탱크는 0~6400좌표 사이 2000~4500정도에 랜덤하게 위치된다. 플레이어는 R키로 적의 위치를 조명으로 찾을 수 있어 게임을 깨는데에는 문제가 없다. 만약 게임을 승리하거나 패배하면 엔딩 씬으로 넘어가며 게임이 종료된다.

2. 실행 결과와 조작법

<이동 관련>

- 방향 키 (좌, 우) : 좌/ 우 방향키를 누르면 플레이어 탱크의 몸체를 각각 왼쪽 회전, 오른쪽 회전
- 방향 키 (상, 하) : 상 / 하 방향키를 누르면 앞으로 / 뒤로 이동. (만약, 회전하면서 이동하고 싶다면 상, 하 방향키를 먼저 누르고 좌/ 우 방향키 누르면 됨.)
- A / D : 각각 A, D키는 플레이어의 Turret방향을 y축기준 회전하여 좌/우 방향으로 회전 시킨다.(조준점도 같이 움직임)
- W / S : 각각 W, S키는 플레이어의 Poshin방향을 x축기준 회전하여 상 / 하 방향으로 이동시킨다 (조준점도 같이 움직임)
- 좌 SHIFT : 일단 진흙 모드가 아닐 때(10초 간격으로 진흙/정상 맵이 번갈아 나타남) 좌 쉬프트를 누르고 이동시 더 빠르게 움직인다.

<기능 관련>

- Q : Q는 플레이어 탱크를 단발모드로 쏘게 하여 단발 모드일 때 카메라가 미사일을 따라가 줌인 효과를 볼 수 있다. (탱크의 미사일 쏘는 모드는 나가고 있는 미사일이 없어야만 적용된다.) -> 좌측상단에 UI로 단발/ 연발 모드인지 확인 가능. 단발 모드에서는 플레이어를 줌인 효과중인 상태에서는 움직이지 못하도록 하였고 회전은 가능하다.
- E : E는 플레이어 탱크를 연발 모드로 쏘게 하여 0.3초마다 한발 씩 나가게 기능 구현하였다. (꼭 누르면 일정시간마다 나간다) 마찬가지로 탱크의 미사일이 현재 발사 중인 미사일이 없어야만 서로 모드를 바꿀 수 있다. 마찬가지로 UI로 확인 가능. 플레이어가 연발모드에서는 이동하면서 연속적으로 미사일을 날릴 수 있다.

- SPACE : 스페이스바는 총알을 발사시킨다. 만약 발사 중인 총알이 있다면
- R: R키는 누르면 2.5초 동안 살아있는 적 객체의 위치를 스팟 라이트로 파란색으로 나타나며, 누른 시점부터 5초 클타임이 있어 UI로 확인 가능하다.
- F : 조준점은 기본적으로 탱크의 포신의 방향과 일치하도록 빌보드로 만들었는데, 조준점이 총 3가지 모드가 있다. 하나는 점선형태와 조준점이 모두 그려지는, 다른 하나는 조준점만 그려지는, 마지막은 점선 궤적만 그려지는 조준점이다 F를 한번씩 누를때마다 모드가 변경된다.

[실행 결과]

1) 시작 씬



2) 플레이 씬



[게임 시작 시, 위에 남은 탱크 수, 남은 시간 카운트 다운]



[단발 모드로 총알 쏘았을 때]



[R키를 눌렀을 때 적 주위 위치 나타내고 쿨타임 UI표시 , 연발모드로 바꿨을 때 UI변경]



[엔딩 씬 - 졌을 때]

3. 자료구조 알고리즘, 구현 내용, 추가적으로 구현한 내용.

1. Terrain

Terrain Input Layout: 내가 기반을 짠 프로젝트는 Terrain이라는 오브젝트가 없었습니다. 그래서 처음에 Terrain이라는 오브젝트를 추가하는 것부터 시작했다. Terrain을 추가하기 위해서는 Mesh부터 만들어야 했는데, Mesh에서 정점의 구조를 살펴보았다. 제가 사용한 코드에서는 CVertex 기본 정점에 Position 값만 있는 클래스, CDiffusedVertex Color 값이 추가된 형태로 CVertex를 상속받는 클래스가 있었다. 하지만 CTexturedShader에서 설명하는 InputLayout에 해당하는 정점 자료형이 CDiffusedTexturedVertex이었는데, 처음에 TexCoord하나만 가지고 있는 형태였다. 그렇기 때문에 3D게임프로그래밍1 때 사용했던 지형 메쉬코드를 가져와 수정하였는데, 처음에는 다중 텍스처로 혼합하는 것이 아닌 단일 텍스처로 지형을 띄워 로드하는 목표로 시작하여 CDiffusedTexturedVertex는 UV좌표 하나를 가지게끔 만들었다. 최종적으로 현재는 텍스처 uv좌표를 두개 가지는 형태로 바꾸었는데, 교수님이 주신 코드를 보니 똑같이 변경하였음을 확인 할 수 있다.

```
void ReleaseUploadBuffers();

protected:
    ID3D12Resource          *m_pd3dVertexBuffer = NULL;
    ID3D12Resource          *m_pd3dVertexUploadBuffer = NULL;

    ID3D12Resource          *m_pd3dIndexBuffer = NULL;
    ID3D12Resource          *m_pd3dIndexUploadBuffer = NULL;

    D3D12_VERTEX_BUFFER_VIEW m_d3dVertexBufferView;
    D3D12_INDEX_BUFFER_VIEW  m_d3dIndexBufferView;

    D3D12_PRIMITIVE_TOPOLOGY m_d3dPrimitiveTopology = D3D_PRIMITIVE_TOPOLOGY_TRIANGLELIST;
    UINT                     m_nSlot = 0;
    UINT                     m_nVertices = 0;
    UINT                     m_nStride = 0;
    UINT                     m_nOffset = 0;

    UINT                     m_nIndices = 0;
    UINT                     m_nStartIndex = 0;
    int                      m_nBaseVertex = 0;

public:
    virtual void Render(ID3D12GraphicsCommandList *pCmdList, ID3D12CommandAllocator *pCmdAllocator);
```

➔ 교수님이 주신 CMesh()

```
protected:
    int m_nVertices = 0;
    UINT m_nStride = 0;
    XMFLAOT3* m_pxmF3Positions = NULL;

    ID3D12Resource* m_pd3dPositionBuffer = NULL;
    ID3D12Resource* m_pd3dPositionUploadBuffer = NULL;
    D3D12_VERTEX_BUFFER_VIEW m_d3dPositionBufferView;

    int m_nSubMeshes = 0;
    int* m_pnSubSetIndices = NULL;
    UINT** m_ppnSubSetIndices = NULL;

    ID3D12Resource** m_ppd3dSubSetIndexBuffers = NULL;
    ID3D12Resource** m_ppd3dSubSetIndexUploadBuffers = NULL;

    UINT m_nIndices = 0;
    ID3D12Resource* m_pd3dIndexBuffer = NULL;
    ID3D12Resource* m_pd3dIndexUploadBuffer = NULL;
    D3D12_INDEX_BUFFER_VIEW m_d3dIndexBufferView;
    D3D12_INDEX_BUFFER_VIEW* m_pd3dSubSetIndexBufferViews = NULL;
```

➔ 사용한 프로젝트 CMesh()

- 나는 사용하던 프로젝트에 Terrain을 추가하였기 때문에 Mesh 객체가 다르게 이루어져 있음을 알 수 있다. 정점버퍼를 설명하는 VertexBuffer와 PositionBuffer는 이름만 다를 뿐 알 수 있고, 내가 사용한 프로젝트는 SubMesh를 설명하는 인덱스들과 그러한 인덱스를 표현하는 인덱스 버퍼가 이중포인터로 배열로 이루어져 있음을 알 수 있다. 단순한 인덱스 버퍼 포인터는 내가 추가한 것인데 Render 등에서 따로 처리하지 않으므로 기능이 없는 코드이고 오로지 SubSetIndexBuffers 배열로 Rendering을 처리하고 있는 프로젝트이다.

```
m_nSubMeshes = 1;
m_pnSubSetIndices = new int[m_nSubMeshes];
m_ppnSubSetIndices = new UINT * [m_nSubMeshes];

m_ppd3dSubSetIndexBuffers = new ID3D12Resource * [m_nSubMeshes];
m_ppd3dSubSetIndexUploadBuffers = new ID3D12Resource * [m_nSubMeshes];
m_pd3dSubSetIndexBufferViews = new D3D12_INDEX_BUFFER_VIEW[m_nSubMeshes];

m_pnSubSetIndices[0] = ((nWidth * 2) * (nLength - 1)) + ((nLength - 1) - 1);
m_ppnSubSetIndices[0] = new UINT[m_pnSubSetIndices[0]];
```

그렇기 때문에 이렇게 동적할당을 통해 하나의 인덱스 버퍼를 사용하더라도 불편하지만 해주어야 한다.

```

void CMesh::Render(ID3D12GraphicsCommandList *pd3dCommandList)
{
    pd3dCommandList->IASetPrimitiveTopology(m_d3dPrimitiveTopology);
    pd3dCommandList->IASetVertexBuffers(m_nSlot, 1, &m_d3dVertexBufferView);
    if (m_pd3dIndexBuffer)
    {
        pd3dCommandList->IASetIndexBuffer(&m_d3dIndexBufferView);
        pd3dCommandList->DrawIndexedInstanced(m_nIndices, 1, 0, 0, 0);
    }
    else
    {
        pd3dCommandList->DrawInstanced(m_nVertices, 1, m_nOffset, 0);
    }
}

```

```

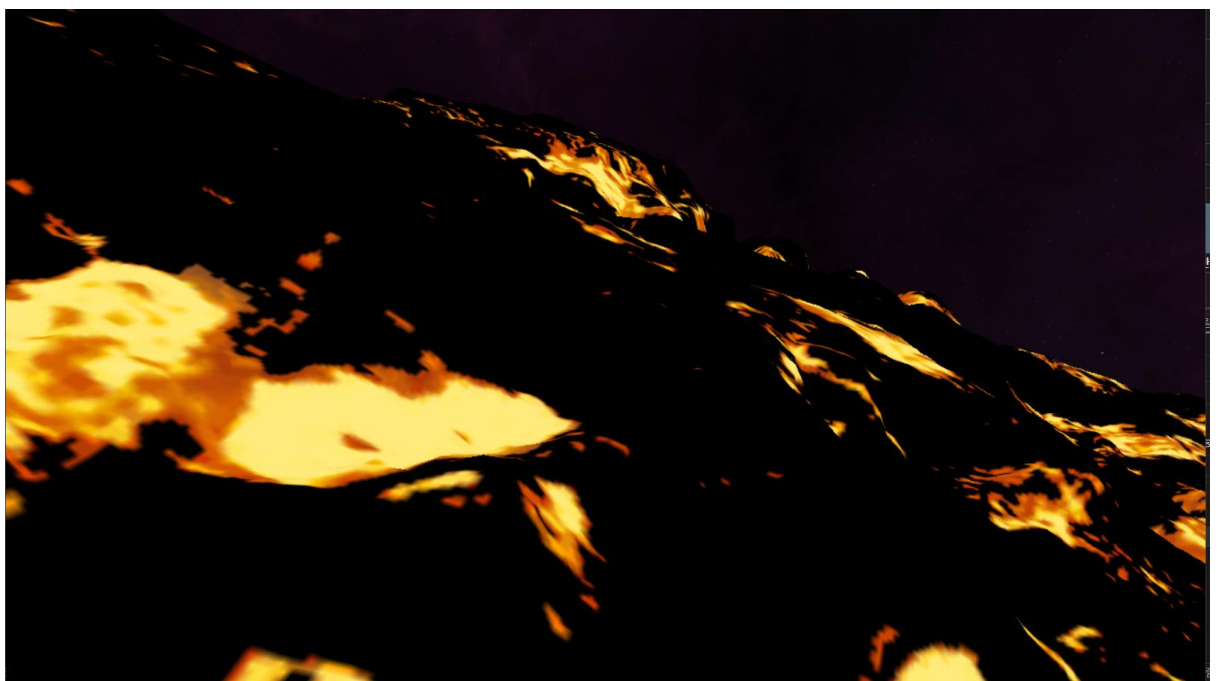
void CMesh::Render(ID3D12GraphicsCommandList* pd3dCommandList, int nSubSet)
{
    pd3dCommandList->IASetPrimitiveTopology(m_d3dPrimitiveTopology);

    pd3dCommandList->IASetVertexBuffers(m_nSlot, 1, &m_d3dPositionBufferView);

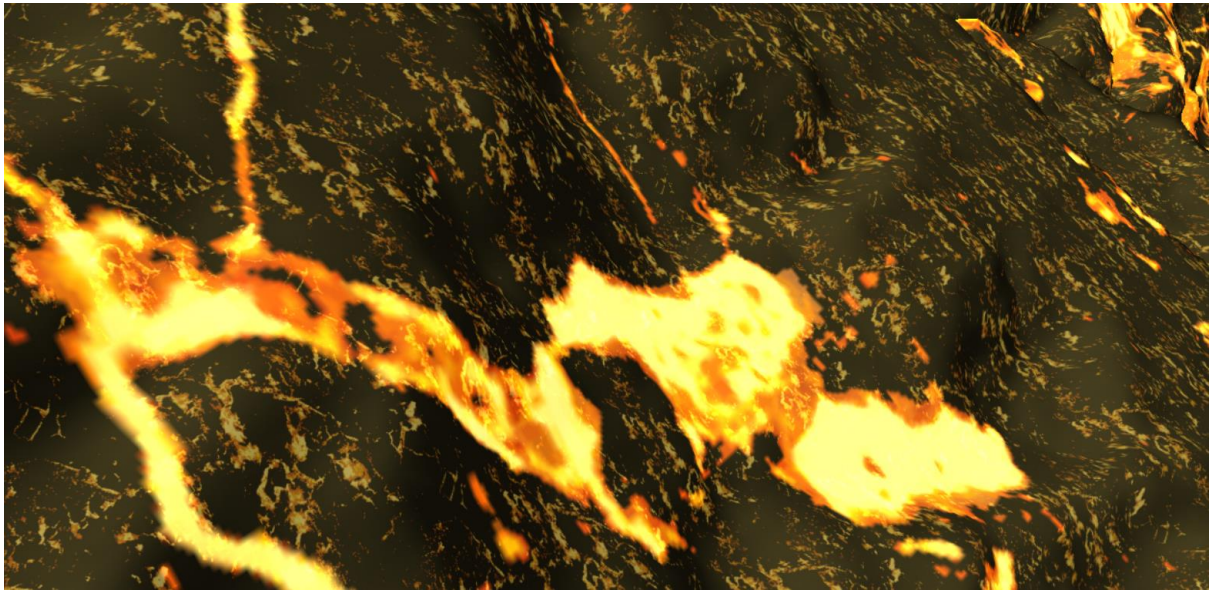
    if ((m_nSubMeshes > 0) && (nSubSet < m_nSubMeshes))
    {
        pd3dCommandList->IASetIndexBuffer(&(m_pd3dSubSetIndexBufferViews[nSubSet]));
        pd3dCommandList->DrawIndexedInstanced(m_nSubSetIndices[nSubSet], 1, 0, 0, 0);
    }
    else
    {
        pd3dCommandList->DrawInstanced(m_nVertices, 1, m_nOffset, 0);
    }
}

```

서브 메쉬를 그릴 수 있도록 subset으로 인덱스를 나누어 렌더링을 처리하는 차이점.



[최초에 Terrain을 띄우고 단일 텍스처로 로드 했을 때.]



➔ 두번째 나아가서 Texture 두개를 가지고 PS에서 Saturate함수로 혼합한 결과

```
//UINT ncbElementBytes = ((sizeof(CB_GAMEOBJECT_INFO) + 255) & ~255); //256의 배수

CTexture* pTerrainTexture = new CTexture(1, RESOURCE_TEXTURE2D, 0, 1);

pTerrainTexture->LoadTextureFromDDSFile(pd3dDevice, pd3dCommandList, L"Image/Lava(Emissive).dds", RESOURCE_TEXTURE2D, 0);

CTexture* pTerrainTextureDetail = new CTexture(1, RESOURCE_TEXTURE2D, 0, 1);
pTerrainTextureDetail->LoadTextureFromDDSFile(pd3dDevice, pd3dCommandList, L"Image/Stone_material.dds", RESOURCE_TEXTURE2D, 0);

CTerrainShader* pTerrainShader = new CTerrainShader();
pTerrainShader->CreateShader(pd3dDevice, pd3dCommandList, pd3dGraphicsRootSignature);
pTerrainShader->CreateShaderVariables(pd3dDevice, pd3dCommandList);

//서술자 힙을 생성하고 -> 그를 채울 뷰들을 만들고 -> 루트시그니처에 바인딩
pTerrainShader->CreateCbvSrvDescriptorHeaps(pd3dDevice, 0, 2);
// 스카이박스 씬에서는 게임오브젝트를 32비트 상수로 하여 자원을 한바이트마다 세세하게 설정가능. 낭비 자원 체크 안해도됨.
// 터레인 씬에서는 게임오브젝트를 뷰로 하여 RangeType으로 나누어 또 세세하게 경매줄 ->공부수필요

//여기서 우리가 셰이더에 건네줄 상수버퍼뷰를 만든다.
pTerrainShader->CreateShaderResourceViews(pd3dDevice, pTerrainTexture, 0, 11);
pTerrainShader->CreateShaderResourceViews(pd3dDevice, pTerrainTextureDetail, 0, 11);

OMaterial* pTerrainMaterial = new OMaterial();
pTerrainMaterial->SetShader(pTerrainShader);
pTerrainMaterial->SetTexture(pTerrainTexture);
SetMaterial(0, pTerrainMaterial);

OMaterial* pTerrainDetailMaterial = new OMaterial();
pTerrainDetailMaterial->SetShader(pTerrainShader);
pTerrainDetailMaterial->SetTexture(pTerrainTexture);
SetMaterial(1, pTerrainDetailMaterial);
```

Terrain 생성자인데 기존 프로젝트와 크게 다른 것은 씬에서 SRV CBV 디스크립터 힙을 만들지 않고 Shader.cpp에 배치하여 각 셰이더에서 SRV디스크립터 힙을 만든다는 점, 또한 현재 조금 무식한 방법이지만 텍스처 2개를 활용하므로 텍스처 배열을 사용해도 되지만, 그냥 Material을 두개 사용하여 각각 텍스처를 하나씩 가지고 있게 하였다. 사실 텍스처 배열을 활용하는 방법을 잘 몰라서 이렇게 하였는데, 교수님이 주신 프로젝트를 보고 현재는 수정한 상태이다. 또 다른 점은 이 프로젝트에서는 Material 객체가 Shader도 가지게 되어있어 만약 오브젝트가 재질이 많다면 저렇게 SetMaterial에 인자를 다르게 하여 재질의 수 인덱스를 정할 수 있다.

→ Terrain Shader

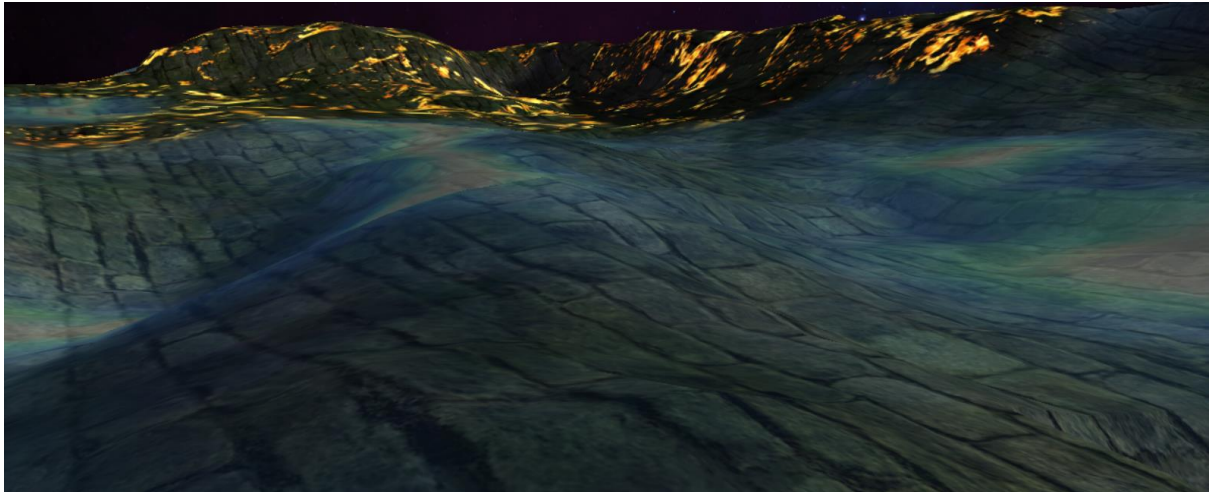
- Terrain Shader는 딱히 기능은 없고 제일 큰 기능은 정점에 대해 설명하는 InputLayout이다 정점에 대한 설명과 구조를 우리는 현재 POSITION, COLOR, UV 좌표 2개를 가지도록 하였기 때문에 이에 대한 설명을 적었다. 다음에는 NORMAL을 추가해 조명계산도 추가해보면 좋을 것 같다. 또한 HLSL에 기재된 VSTerrain, PSTerrain을 사용할 수 있도록 셰이더 만드는 코드를 바꾸어주었다. 그리고 PSO 객체를 만드는 코드도 기존 프로젝트와 살짝 다른데 PipelineStateDesc 객체가 따로 멤버 변수로 지니도록 되어있어 생성해주고 CreateShader를 호출하도록 모든 셰이더클래스에 구조를 만들었다.

→ Terrain Texture RootSignature

```
pd3dDescriptorRanges[8].RangeType = D3D12_DESCRIPTOR_RANGE_TYPE_SRV;  
pd3dDescriptorRanges[8].NumDescriptors = 1;  
pd3dDescriptorRanges[8].BaseShaderRegister = 14; //t14: gtxtTerrainBaseTexture  
pd3dDescriptorRanges[8].RegisterSpace = 0;  
pd3dDescriptorRanges[8].OffsetInDescriptorsFromTableStart = D3D12_DESCRIPTOR_RANGE_OFFSET_APPEND;  
  
Texture2D gtxtTerrainBaseTexture : register(t14);  
//Texture2D gtxtTerrainDetail : register(t15);  
  
float4 PSTerrain(VS_TERRAIN_OUTPUT input) : SV_TARGET  
{  
    float4 cBaseTexColor = gtxtTerrainBaseTexture.Sample(gssWrap, input.uv);  
    float4 cDetailTexColor = gtxtTerrainBaseTexture.Sample(gssWrap, input.uv1);  
  
    float4 cColor = saturate(input.color*0.3f+(cBaseTexColor+cDetailTexColor*0.5f));  
    //float4 cColor = cBaseTexColor;  
    return(cColor);  
}
```

- 또 기존 프로젝트와 크게 다른점은 내가 사용한 프로젝트는 월드 행렬을 루트상수로 넘겨주기 때문에 오브젝트 생성자 마다 CBV는 만들어줄 필요가 딱히 없었다. 그렇기 때문에 SRV디스크립터 힙과 뷰만 잘 관리해주면 되었다. 그렇기 때문에 Terrain과 관련해서 RootSignature를 바꿔줘야하는 것은 모두 Texture과 관련되어있었는데, 텍스처 2개를 혼합할 때 잘 몰라서 같은 레지스터 번호에 텍스처만 다르게 하여 Material을 두개 가지고 같은 레지스터에 꽂아 넣어 루트 파라미터는 하나만 차지하도록 하고 Descriptor개수도 하나로 하였다. 그렇기 때문에 셰이더 코드에서도 같은 변수로 그냥 gtxtTerrainBaseTexture로 사용하였다. 이게 문제가 되는지는 잘 모르겠지만 일단 통과가 되어 사용하였다.

[세번째 변경된 Terrain]



현재는 교수님이 주신 프로젝트에서 다양한 텍스처를 혼합한 결과물의 Terrain의 형태로 자리 잡았다. 아직 괜찮은 조합을 찾진 못해서 수정중이지만 코드는 완성되었고 텍스처만 바꾸면 될 것 같다. 루트 시그니처는 텍스처 배열을 사용할 수 있도록 NumDescriptor를 마찬가지로 5개로 바꾸었고 Range는 하나를 사용하였다. 셰이더 내부 코드도 다시 바꾸어 기존의 Saturate함수를 lerp 함수로 연산하도록 바꾸었고 Sampler는 마찬가지로 Wrap형태로 만들었다. 전에는 테스트 차원에서 샘플러도 하나 추가하여 Wrap말고도 Clamp형태를 사용할 수 있는 샘플러도 추가하였다.

```
// CTexturedVertex *pVertices = new CTexturedVertex[m_nVertices];
CDiffusedTexturedVertex* pVertices = new CDiffusedTexturedVertex[m_nVertices];

CHightMapImage* pHightMapImage = (CHightMapImage*)pContext;
int cxHeightMap = pHightMapImage->GetHeightMapWidth();
int czHeightMap = pHightMapImage->GetHeightMapLength();

float fHeight = 0.0f, fMinHeight = +FLT_MAX, fMaxHeight = -FLT_MAX;
for (int i = 0, z = zStart; z < (zStart + nLength); z++)
{
    for (int x = xStart; x < (xStart + nWidth); x++, i++)
    {
        fHeight = OnGetHeight(x, z, pContext);
        pVertices[i].m_xmf3Position = XMFLAT3((x * m_xmf3Scale.x), fHeight, (z * m_xmf3Scale.z));
        pVertices[i].m_xmf4Diffuse = Vector4::Add(OnGetColor(x, z, pContext), xmf4Color);
        pVertices[i].m_xmf2TexCoord = XMFLAT2(float(x) / float(cxHeightMap - 1), float(czHeightMap - 1 - z) / float(czHeightMap - 1));
        pVertices[i].m_xmf2TexCoord0 = XMFLAT2(float(x) / float(m_xmf3Scale.x * 0.5f), float(z) / float(m_xmf3Scale.z * 0.5f));
        // Detail 텍스처로 경사로 늘어짐에따른 텍스처 디테일 보완시키는.
        if (fHeight < fMinHeight) fMinHeight = fHeight;
        if (fHeight > fMaxHeight) fMaxHeight = fHeight;
    }
}
```

-> 현재 지형은 컬러값을 받고 있는데, 이 컬러 값은 텍스처를 사용한다면 사용되지 않는다. 그렇기 때문에 조명을 받을 수 있도록 Normal값으로 바꾸는 작업을 수행하였다.

```

// CTexturedVertex* pVertices = new CTexturedVertex[m_nVertices];
CNormalTexturedVertex* pVertices = new CNormalTexturedVertex[m_nVertices];

CHightMapImage* pHeightMapImage = (CHightMapImage*)pContext;
int cxHeightMap = pHeightMapImage->GetHeightMapWidth();
int czHeightMap = pHeightMapImage->GetHeightMapLength();

float fHeight = 0.0f, fMinHeight = +FLT_MAX, fMaxHeight = -FLT_MAX;
for (int i = 0, z = zStart; z < (zStart + nLength); z++)
{
    for (int x = xStart; x < (xStart + nWidth); x++, i++)
    {
        fHeight = OnGetHeight(x, z, pContext);
        pVertices[i].m_xmf3Position = XMFLAT3((x * m_xmf3Scale.x), fHeight, (z * m_xmf3Scale.z));
        pVertices[i].m_xmf3Normal = pHeightMapImage->GetHeightMapNormal((x * m_xmf3Scale.x), (z * m_xmf3Scale.z));
        pVertices[i].m_xmf2TexCoord = XMFLAT2(float(x) / float(cxHeightMap - 1), float(czHeightMap - 1 - z) / float(czHeightMap - 1));
        pVertices[i].m_xmf2TexCoord0 = XMFLAT2(float(x) / float(m_xmf3Scale.x * 0.5f), float(z) / float(m_xmf3Scale.z * 0.5f));
        // Detail 텍스처로 경사로 늘어짐에따른 텍스처 디테일 보완시키는.
        if (fHeight < fMinHeight) fMinHeight = fHeight;
        if (fHeight > fMaxHeight) fMaxHeight = fHeight;
    }
}

```

-> Normal 값을 이용하여 TerrainShader에 적용하였다.



지형은 10초 간격으로 진흙 재질, 기본 잡초 재질의 텍스처가 번갈아 가며 BaseTerrainTexture이 되어 진흙 색을 띄게 되며 이때는 플레이어의 기본 이동속도와 Accelerate가 제한된다. 그러므로 이 때는 적의 위치를 찾는데에 전략 세우고, 기본 지형 모드일 때 그 위치로 Accelerate하여 다가가는 목적으로 게임을 구성하였다. 텍스처는 둘다 알파텍스처를 적용해 해당 알파가 투명한 곳은 디테일 텍스처를 적용해 흰색 무늬를 띄게 하였다. 모두 텍스처를 5개씩 사용하고 모드마다 Base텍스처만 번갈도록 하였다. 시간은 셰이더에서 받기 때문에 그 자원을 재활용하여 시간마다 번갈아가면서 텍스처를 적용하도록 하였다.



[두가지의 지형과 물 재질]

2. SKYBOX

- 스카이 박스 구현

-> 스카이박스도 메쉬를 직접 정점 버퍼에 큐브 형태로 만들어 큐브 텍스처를 적용하는 방법으로 구현하였다. 메쉬는 간단하게 텍스처를 입히지만 UV좌표가 필요없는 형태였다. 스카이박스는 CUBE텍스처를 사용하는데 알아서 매핑되는 형태이므로 UV좌표가 필요없다.

```
CSkyBox::CSkyBox(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList, ID3D12RootSignature* pd3dRootSignature)
{
    CSkyBoxMesh* pSkyBoxMesh = new CSkyBoxMesh(pd3dDevice, pd3dCommandList, 20.0f, 20.0f, 20.0f);
    SetMesh(pSkyBoxMesh);

    CreateShaderVariables(pd3dDevice, pd3dCommandList);

    CTexture* pSkyBoxTexture = new CTexture(1, RESOURCE_TEXTURE_CUBE, 0, 1);
    pSkyBoxTexture->LoadTextureFromDDSFile(pd3dDevice, pd3dCommandList, L"SkyBox/SkyBox_1.dds", RESOURCE_TEXTURE_CUBE);

    CSkyBoxShader* pSkyBoxShader = new CSkyBoxShader();
    pSkyBoxShader->CreateShader(pd3dDevice, pd3dCommandList, pd3dRootSignature);
    pSkyBoxShader->CreateShaderVariables(pd3dDevice, pd3dCommandList);

    pSkyBoxShader->CreateCbvSrvDescriptorHeaps(pd3dDevice, 0, 1);
    pSkyBoxShader->CreateShaderResourceViews(pd3dDevice, pSkyBoxTexture, 0, PARAMETER_SKYBOX_CUBE_TEXTURE);

    CMaterial* pSkyBoxMaterial = new CMaterial();
    pSkyBoxMaterial->SetTexture(pSkyBoxTexture);
    pSkyBoxMaterial->SetShader(pSkyBoxShader);

    SetMaterial(0, pSkyBoxMaterial); //변경 SetMaterial(0, pSkyBoxMaterial);
}

CSkyBox::~CSkyBox()
```

스카이박스도 구조를 내가 사용하는 프로젝트 형태에 맞게 구현하였다. 스카이박스도 셰이더를 개별적으로 만들어 주어 스카이박스에 대한 처리를 할 수 있도록 하였다.

- SkyBox Shader

➔ 스카이박스 셰이더에서 정점 설명은 Pos값 하나만 들고있게 하여도 된다. 텍스처는 알아서 매핑되기 때문이다. 또한 중요한건 CreateDepthStencilState를 하여 PSO에서 Depth값을 사용하지 않도록 변경해주어야한다.

```

struct VS_SKYBOX_CUBE_MAP_INPUT
{
    float3 position : POSITION;
};

struct VS_SKYBOX_CUBE_MAP_OUTPUT
{
    float3 positionL : POSITION;
    float4 position : SV_POSITION;
};

VS_SKYBOX_CUBE_MAP_OUTPUT VSSkyBox(VS_SKYBOX_CUBE_MAP_INPUT input)
{
    VS_SKYBOX_CUBE_MAP_OUTPUT output;

    output.position = mul(mul(mul(float4(input.position, 1.0f), gmtxGameObject), gmtxView), gmtxProjection);
    output.positionL = input.position;

    return(output);
}

TextureCube gtxtSkyCubeTexture : register(t13);
SamplerState gssClamp : register(s1);

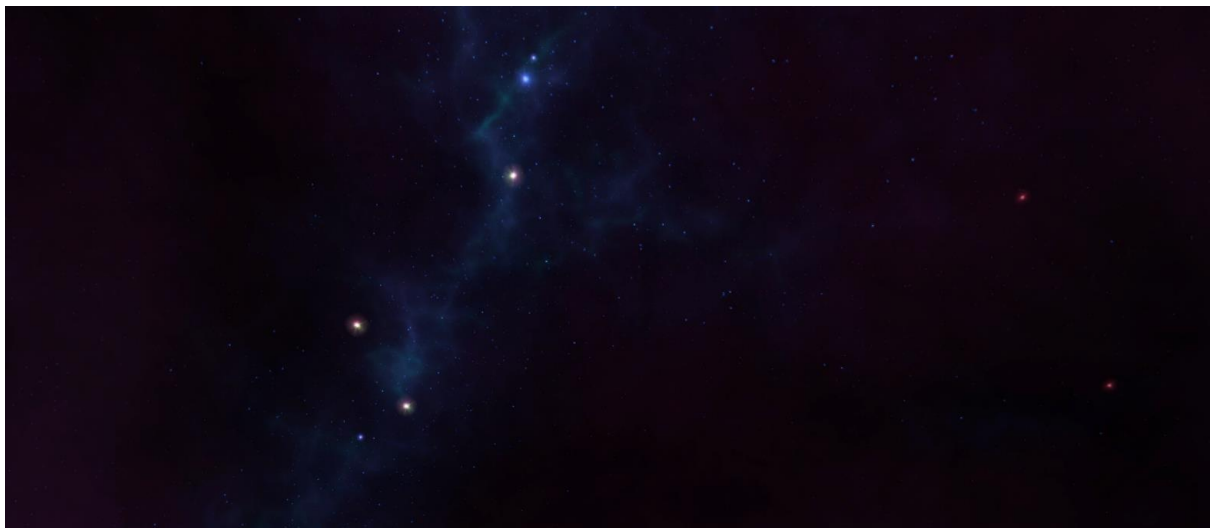
float4 PSSkyBox(VS_SKYBOX_CUBE_MAP_OUTPUT input) : SV_TARGET
{
    float4 cColor = gtxtSkyCubeTexture.Sample(gssClamp, input.positionL);

    return(cColor);
}

```

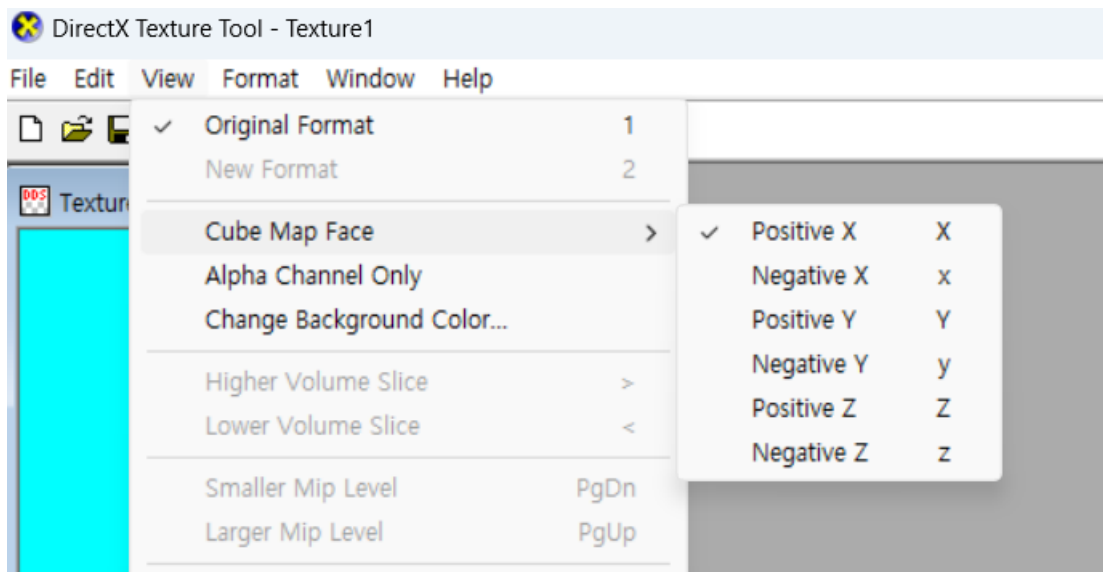
셰이더 코드에서는 샘플러 두번째를 사용하여 Clamp 형태로 결과를 만들어주었다.

➔ 결과

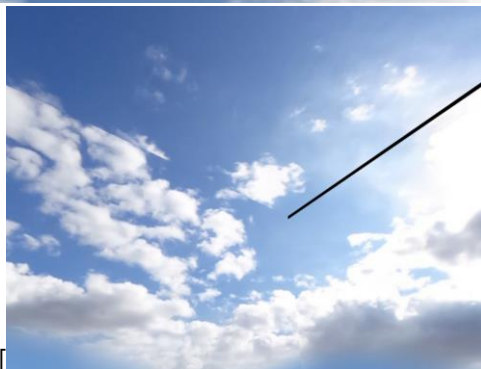
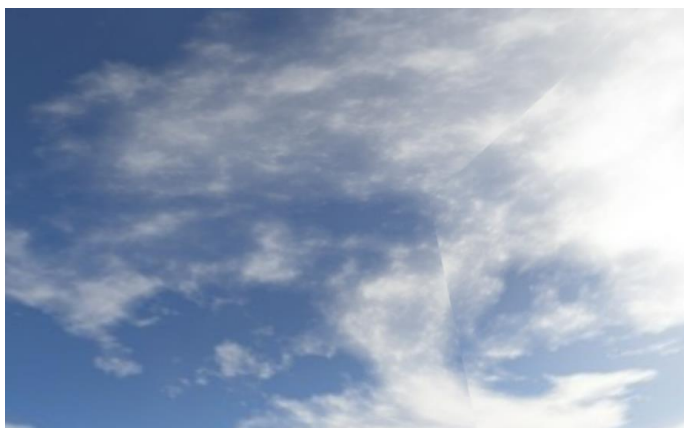


➔ 스카이박스 텍스처 만들기

- 스카이박스 텍스처를 인터넷에서 $x, -x, y, -y, z, -z$ 면에 해당하는 텍스처를 구하여 큐브 텍스처로 dds파일 추출하는 것도 시도해보았다.



이 툴을 활용하여 여러 스카이박스 텍스처를 만들어 로드도 시도해보았다. 인터넷에서 하나의 텍스처를 올려서 큐브맵 형태로 만들 때 경계가 뚜렷하지 않아 결과적으로 실패하였다. 그래서 $+x, -x, +y, -y, +z, -z$ 면에 해당하는 텍스처를 제공하는 사이트에서 추출해와 만들어 보았다.

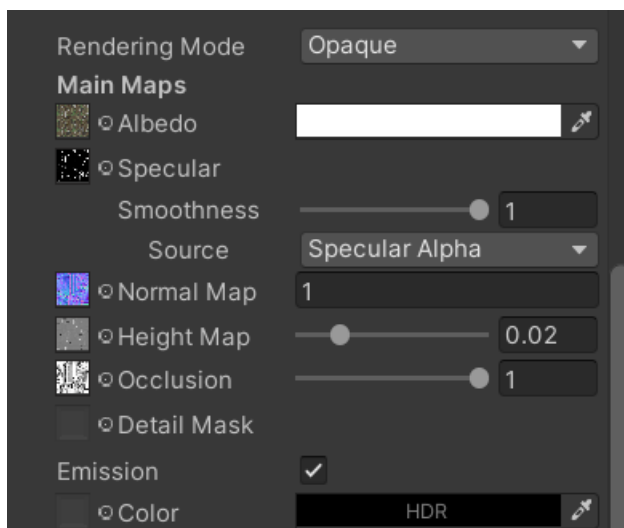


[잘 못 만든 결과들]



-> 정상적으로 만든 스카이박스를 적용해보았다. 하지만 너무 밝아서 PSSkyBox 코드에서 특정 고정 색상값 회색 빛 `float4(0.5,0.5,0.5,1.0)` 과 선형보간하도록 만들었다.

3. Player



```

void CTankPlayer::PrepareAnimate()
{
    m_pWheel[0] = FindFrame("Cube.000");
    m_pWheel[1] = FindFrame("Cube.001");
    m_pWheel[2] = FindFrame("Cube.003");
    m_pWheel[3] = FindFrame("Cube.004");
    m_pWheel[4] = FindFrame("Cube.005");
    m_pWheel[5] = FindFrame("Cube.006");
    m_pWheel[6] = FindFrame("Cube.007");
    m_pWheel[7] = FindFrame("Cube.008");
    m_pWheel[8] = FindFrame("Cube.009");
    m_pWheel[9] = FindFrame("Cube.010");
    m_pWheel[10] = FindFrame("Cube.011");
    m_pWheel[11] = FindFrame("Cube.012");
    m_pWheel[12] = FindFrame("Cube.013");
    m_pWheel[13] = FindFrame("Cube.014");
    m_pWheel[14] = FindFrame("Cube.015");
    m_pWheel[15] = FindFrame("Cube.016");
    m_pTurret = FindFrame("Cube.017");
    m_pPushin = FindFrame("Cube.018");
}

```

```

<SpecularColor>: 0.2 0.2 0.2 1
0.5 <Glossiness>:
<SpecularHighlight>: 1
<GlossyReflection>: 1
@MainTank_Base_Color <AlbedoMap>:
@MainTank_Metallic <SpecularMap>:
@MainTank_Normal <NormalMap>:
null <EmissionMap>:

```

플레이어는 탱크 모델을 유니티에서 불러와 텍스처를 입히는 형태로 만들었다. 각 텍스처는 모델에서 주어진 텍스처를 dds 포맷으로 뽑아 다시 재질에 넣어서 바꿔주었다. 플

레이어의 바퀴들과 포신 및 터렛은 FindFrame으로 유니티에서 설정한 자식관계를 로드 해주었다.

```
public:
    float m_fBulletEffectiveRange = 800.0f;

    CBulletObject* pBulletObject = NULL;
    //CGameObject* m_pMainMotorFrame = NULL;
    virtual void Update(float fTimeElapsed)override;

    array<CGameObject*,16> m_pWheel{};
    int m_nWheels = 16;
    bool is_RotateWheel = false; // 움직(회전 포함)이고 있는지
    bool RotateWheel_Forward = true; //바퀴 굴러가도록 false면 animate에서 뒤로 굴러가게
    bool is_Going = false; // 좌우 누를때 판단
    bool Fired_Bullet = false; //총 쏘면 반응위해

    float turret_rotate_value = 0.0f; //시간에 비례한 회전량을 통해 회전 시키게
    float poshin_rotate_value = 0.0f; // 이는 a d키 누르면 썬에서 조작할.

    bool poshin_direction = false; //

    CGameObject* m_pTurret = NULL;
    CGameObject* m_pPoshin = NULL;
    virtual void Rotate(float x, float y, float z)override;
    virtual CCamera* ChangeCamera(DWORD nNewCameraMode, float fTimeElapsed);
    virtual void OnPrepareRender();

    void OnPlayerUpdateCallback(float fTimeElapsed);
    void OnCameraUpdateCallback(float fTimeElapsed);

    void UpdateTankPosition(float fTimeElapsed);

    void FireEffect(float fTimeElapsed);
    float FireEffectTimeElapsed = 0.f;
    float FireEffectDuration = 0.15f;

    void FireBullet(CGameObject* pLockedObject);
    float m_fdelrot = 1.0;
    BoundingOrientedBox xoobb = BoundingOrientedBox(GetPosition(), XMFLOAT3(15.0, 10.0, 30.0), XMFLOAT4(0.0, 0.0, 0.0, 1.0));

    virtual void Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera);
```

플레이어는 기존의 프로젝트에 CAirplanePlayer이 있었지만 TankPlayer를 추가하여 기능을 모두 바꾸었다. Tank플레이어는 총알을 가지도록 만들었고, 바운딩박스, 또한 포신 및 터렛을 회전 시키기 위한 값들도 멤버로 가지게 하였다. 추가적으로 지형에서 플레이어 위치를 업데이트 하기 위한 Update함수와 Camera도 Update하게 하였다. TankPlayer는 기본 카메라가 3인칭 카메라로 설정되어있게 하였다.

```
D3D12_INPUT_LAYOUT_DESC CPlayerShader::CreateInputLayout()
{
    UINT nInputElementDescs = 5;
    D3D12_INPUT_ELEMENT_DESC* pd3dInputElementDescs = new D3D12_INPUT_ELEMENT_DESC[nInputElementDescs];

    pd3dInputElementDescs[0] = { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 };
    pd3dInputElementDescs[1] = { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 1, 0, D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 };
    pd3dInputElementDescs[2] = { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 2, 0, D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 };
    pd3dInputElementDescs[3] = { "TANGENT", 0, DXGI_FORMAT_R32G32B32_FLOAT, 3, 0, D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 };
    pd3dInputElementDescs[4] = { "BITANGENT", 0, DXGI_FORMAT_R32G32B32_FLOAT, 4, 0, D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 };

    D3D12_INPUT_LAYOUT_DESC d3dInputLayoutDesc;
    d3dInputLayoutDesc.pInputElementDescs = pd3dInputElementDescs;
    d3dInputLayoutDesc.NumElements = nInputElementDescs;

    return(d3dInputLayoutDesc);
}
```

플레이어는 Albedo 텍스처, Normal 텍스처, Height 텍스처, Metallic 텍스처, Occlusion 텍

스처를 로드하게 하였기 때문에 Normal 매핑을 적용시키기 위해 InputLayout은 위치, UV, 노멀 탄젠트 바이탄젠트 값들을 적용시켰다. 또한 StandardVS 와 PS에서 이 값들을 활용하는 셰이더 코드를 반영시키도록 PlayerShader를 구성하였다.

- Animate

플레이어의 애니메이션은 기본적으로 앞으로 가고 뒤로 가는 것은 앞뒤 방향키로 이루게 했고, 이 동작들을 수행함으로써 모든 바퀴가 앞으로 회전하고 뒤로 회전하게끔 만들었다. 또한 좌우 방향키는 옆으로 회전시키도록 만들어 좌키와 앞키를 누르면 방향을 꺾으면서 앞으로 가게 구현하였다. 이 때 두키를 동시에 누르다가 하나를 떼면 지속적으로 바퀴도 굴러가게 하였다.



-> 텍스처를 입힌 탱크 플레이어 모습, 현재 터렛과 포신을 각각 따로 회전 시킨 모습.



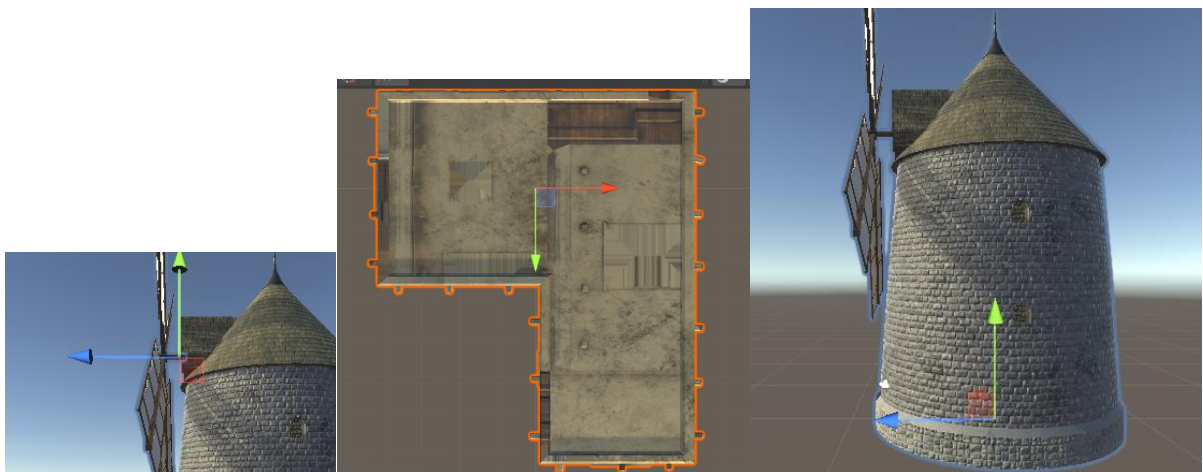
-> 플레이어는 카메라 모드를 총 4가지를 가지도록 카메라를 추가했으며 4번째 추가된 카메라는 탱크의 옆모습을 보도록 하였다.

- 플레이어는 또한 좌 Shift키로 만약 지형이 진흙 상태가 아니라면 가속을 할 수 있고, 조준은 a,d,w,s또는 마우스로 할 수 있다. 플레이어는 건물들과 모두 충돌처리가 되어 있고, 돌 및 선인장에 대해서도 충돌처리가 되어 있다. 적과의 직접적인 충돌처리로 인한 효과는 없으며, 적의 미사일에도 피격 당하지는 않는다. 게임의 목적은 3분내에 최대한 많은 탱크를 찾아 처치하는 것이므로 그에 맞게 플레이어를 만들었다.

4. Buildings

건물들은 총 4가지인데 3가지는 DesertBuilding 형태이고 나머지 1가지는 WindMill 건물이며, 총 20개의 건물을 랜덤한 위치에 만들었다. 각 오브젝트는 유니티에서 불러왔으며 텍스처도 다 입히게 하였다. 텍스처는 Albedo, Metallic, Normal이 있어 마찬가지로 CStandardShader에 포함된 Bitangent 등InputLayout을 사용하도록 상속하였고 CBuildingsShader 와 따로 윈드밀이 회전하도록 Animation을 추가하기 위한 CWindMillShader을 추가하였다.

각 CBuildingsShader, CWindMillShader는 BuildObject에서 Terrain을 받도록하여 씬에서 Terrain을 만들고나서 각 셰이더의 BuildObject를 호출하게 하였다. Terrain을 받아온 이유는 Building을 위치시킬 위치에서의 x,z에 따른 Terrain의 높이를 구하여 해당 높이에 각 건물을 만들기 위함이다.



사진에서 보다시피 각 오브젝트의 축이 각자 개성에 맞게 이상하게 비치 되어있다. 이때문에 SetPosition등을 할 때 유니티에서 0, 0 , 0에 비치했을 때 기준 축에 맞게 비치되는데 건물들과 같은 경우 축이 이상하게 되어있어 조정할 수치들이 많았다. WindMill의 경우 부모 오브젝트의 축이 바닥에 있어 회전하는 날개를 회전하였을 때 공전하는 애니메이션이 처음에 구현되었다. 그렇기 때문에 유니티에서 빈 오브젝트를 만들어 회전 축을 설정하고 그 설정한 축 오브젝트를 날개의 부모로 하여 날개가 회전할 수 있도록 만들어 주었다.



```
void CScene::CheckBuilding_threeByTankCollisions()
{
    XMFLDAT3 pos = m_pPlayer->GetPosition();
    CBuildingShader* pShader = static_cast<CBuildingShader*>(m_ppShaders[1]);
    for (int i = 0; i < 5; i++) {
        if (pShader->m_ppBuildings[i]->m_xmCollision.Intersects(m_pPlayer->m_xmCollision)) {
            //탈크 기포 절반 데미지 4.0f 세로 9.5f 이 건물 기포 08 세로 80 속으로 부터 23쪽 25 2음쪽 65 x양쪽 56 음쪽 42
            if (m_pPlayer->GetPosition().z - 4.f < pShader->m_ppBuildings[i]->GetPosition().z + 25.f && m_pPlayer->GetPosition().z + 4.0f > pShader->m_ppBuildings[i]->GetPosition().z + 25.f) {
                if (m_pPlayer->GetPosition().x - 10.f < pShader->m_ppBuildings[i]->GetPosition().x + 55.f && m_pPlayer->GetPosition().x + 4.f > pShader->m_ppBuildings[i]->GetPosition().x + 55.f) {
                    //0.5f차이로 충돌여부 체크 하는것 9.5* 0.5(0부분으로)
                    if (m_pPlayer->GetPosition().x > pShader->m_ppBuildings[i]->GetPosition().x) {
                        pos.x = pShader->m_ppBuildings[i]->GetPosition().x + 64.5f;
                        m_pPlayer->SetPosition(pos);
                    }
                    else {
                        pos.x = pShader->m_ppBuildings[i]->GetPosition().x - 52.5f;
                        m_pPlayer->SetPosition(pos);
                    }
                }
                else if (m_pPlayer->GetPosition().x - 4.0f < pShader->m_ppBuildings[i]->GetPosition().x + 55.f && m_pPlayer->GetPosition().x + 4.f > pShader->m_ppBuildings[i]->GetPosition().x + 55.f) {
                    if (m_pPlayer->GetPosition().z - 10.0f < pShader->m_ppBuildings[i]->GetPosition().z + 25.f && m_pPlayer->GetPosition().z + 10.f > pShader->m_ppBuildings[i]->GetPosition().z + 25.f) {
                        if (m_pPlayer->GetPosition().z > pShader->m_ppBuildings[i]->GetPosition().z) {
                            pos.z = pShader->m_ppBuildings[i]->GetPosition().z + 34.f;
                            m_pPlayer->SetPosition(pos);
                        }
                        else {
                            pos.z = pShader->m_ppBuildings[i]->GetPosition().z - 74.f;
                            m_pPlayer->SetPosition(pos);
                        }
                    }
                }
            }
            //pShader->m_ppBuildings[i]->collapse = true;
        }
    }
}
```

-> 충돌처리

각 건물들을 충돌처리를 플레이어와 이루어지도록 구현하였는데, 아까 말했듯이 속도 이상하고 가운데에있지 않아 바운딩 박스 수치도 적합한 수치를 조정하는게 어려웠다. 유니티에서 작은 1,1,1길이의 큐브를 만들어 각 건물 모서리간의 길이를 구하여 수치로 환산하여 좌표 길이들을 통해 충돌처리를 구현하였다. 대충 플레이어 위치가 건물에 해당하는 x, z 길이에 들어오고 바운딩박스가 Intersect할 때 플레이어가 오는 위치에 따라 그만큼 x, z등을 밀어 충돌하는 느낌을 주어 건물에 부딪히는 형태로 만들었다. 위 사진에서 보는 건물 같은 경우 'ㄱ'자로 되어있어 여백을 처리하는 것 때문에 더 세밀하게 구간을 나누어 충돌처리를 구현하였지만, 'ㄱ'자 패인부분에 충돌하면 순간이동하는 버그도 있는데 해결을 못하였다. 충돌처리 관련함수는 Scene의 Animate에 넣어 매 프레임마다 확인 하게끔 하였고 그 전에 모든 객체에 대한 바운딩박스를 Update를 하였다.

```
bool CShader::ReturnTerrainHeightDifference(float h1, float h2, void* pContext)
{
    CHeightMapTerrain* pTerrain = (CHeightMapTerrain*)pContext;
    if (pTerrain->GetHeight(h1, h2) < 190.0f) return false;
    if (abs(pTerrain->GetHeight(h1, h2) - pTerrain->GetHeight(h1 + 10, h2)) > 3.0f || abs(pTerrain->GetHeight(h1, h2) - pTerrain->GetHeight(h1, h2 + 10)) > 3.0f || abs(pTerrain->GetHeight(h1, h2) - pTerrain->GetHeight(h1 + 10, h2 + 10)) > 3.0f)
        return false;
    else {
        return true;
    }
}
```

[해당 x,z 좌표기준 terrain 주변 높이값 차가 3보다 큰지 확인하는]

건물들은 모두 랜덤한 위치에 생성하되, 굴곡이 심한 지형에는 위치되지 않게끔 좌표 x,z 를 기준으로 주변 범위 10만큼 떨어진 좌표의 높이가 3보다 큰 경우에는 false를 반환하는 함수를 만들어 평평한 곳에 건물이 위치될 수 있도록 하였다.

5. Camera

```
        break;
    case THIRD_PERSON_CAMERA:
        SetFriction(250.0f);
        SetGravity(XMFLOAT3(0.0f, -250.0f, 0.0f));
        SetMaxVelocityXZ(300.0f);
        SetMaxVelocityY(400.0f);
        m_pCamera = OnChangeCamera(THIRD_PERSON_CAMERA, nCurrentCameraMode);
        m_pCamera->SetTimeLag(1.25f);
        m_pCamera->SetOffset(XMFLOAT3(0.0f, 25.0f, -60.0f));
        m_pCamera->SetPosition(Vector3::Add(m_xmf3Position, m_pCamera->GetOffset()));
        m_pCamera->GenerateProjectionMatrix(1.01f, 50000.0f, ASPECT_RATIO, 60.0f);
        break;
    case LEFT_CAMERA:
        SetFriction(250.0f);
        SetGravity(XMFLOAT3(0.0f, -250.0f, 0.0f));
        SetMaxVelocityXZ(300.0f);
        SetMaxVelocityY(400.0f);
        m_pCamera = OnChangeCamera(LEFT_CAMERA, nCurrentCameraMode);
        m_pCamera->SetTimeLag(1.25f);
        m_pCamera->SetOffset(XMFLOAT3(0.0f, 20.0f, -100.0f));
        m_pCamera->SetPosition(Vector3::Add(m_xmf3Position, m_pCamera->GetOffset()));
        m_pCamera->GenerateProjectionMatrix(1.01f, 50000.0f, ASPECT_RATIO, 60.0f);
        break;
```

카메라는 게임을 시작할 때, 게임이 끝날 때를 위해 카메라 모드를 하나 추가하였다. LEFT_MODE 카메라를 CTankPlayer 클래스에 추가하였다. 이 때는 카메라가 플레이어를 앞에서 바라보도록하여 게임을 시작하거나 끝났을 때의 씬을 위해 이렇게 만들었다. 또한 마지막에 게임이 끝날 때 특정 애니메이션을 구현하기 위해 모드를 추가하였다.

총알을 날릴 때 단발 모드라면, 카메라가 총알을 따라가도록 구현하였는데, 이를 통해 정밀하게 총알을 날리는 것을 체크할 수 있었고, 총알을 따라가면서 적이 맞거나 했을 때 더 생동감이 있는 효과가 있었다.

```

void CThirdPersonCamera::UpdateByBullet(XMFLOAT3& xmf3LookAt, float fTimeElapsed)
{
    CBulletsShader* pBulletShader = static_cast<CBulletsShader*>(m_pPlayer->m_pPlayerShader);
    CTankPlayer* pTankPlayer = static_cast<CTankPlayer*>(m_pPlayer);
    if (m_pPlayer)
    {
        XMFL0AT4x4 xmf4x4Rotate = Matrix4x4::Identity();
        XMFL0AT3 xmf3Right = pBulletShader->m_ppBullets[0]->GetRight();
        XMFL0AT3 xmf3Up = XMFL0AT3(0.f, 1.f, 0.f);
        XMFL0AT3 xmf3Look = pBulletShader->m_ppBullets[0]->GetLook(); // (pTankPlayer->m_pPosin->GetLook());
        // pBulletShader->m_ppBullets[0]->GetLook();
        xmf4x4Rotate._11 = xmf3Right.x; xmf4x4Rotate._21 = xmf3Up.x; xmf4x4Rotate._31 = xmf3Look.x;
        xmf4x4Rotate._12 = xmf3Right.y; xmf4x4Rotate._22 = xmf3Up.y; xmf4x4Rotate._32 = xmf3Look.y;
        xmf4x4Rotate._13 = xmf3Right.z; xmf4x4Rotate._23 = xmf3Up.z; xmf4x4Rotate._33 = xmf3Look.z;

        XMFL0AT3 xmf3Offset = Vector3::TransformCoord(m_xmf3Offset, xmf4x4Rotate);
        XMFL0AT3 xmf3Position = Vector3::Add(pBulletShader->m_ppBullets[0]->GetPosition(), xmf3Offset);
        XMFL0AT3 xmf3Direction = Vector3::Subtract(xmf3Position, m_xmf3Position, false);
        float fLength = Vector3::Length(xmf3Direction);
        xmf3Direction = Vector3::Normalize(xmf3Direction);
        float fTimeLagScale = (m_fTimeLag) ? fTimeElapsed * (1.0f / m_fTimeLag) : 1.0f;
        float fDistance = fLength * fTimeLagScale;
        if (fDistance > fLength) fDistance = fLength;
        if (fLength < 0.01f) fDistance = fLength;
        if (fDistance > 0&&!pBulletShader->m_ppBullets[0]->Collided)
        {
            m_xmf3Position.y += 30.0f*fTimeElapsed;
            m_xmf3Position = Vector3::Add(m_xmf3Position, xmf3Direction, fDistance*5);
            SetLookAt(xmf3LookAt);
        }
    }
}

```

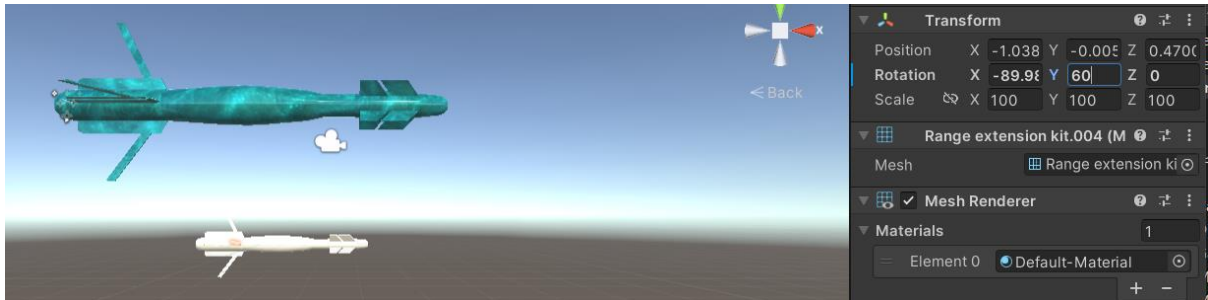
이를 구현하기 위해서 기존에 Player에 있는 Update함수를 뒤엎었어야 했는데, BulletMode를 정하여서 이 BulletMode가 연발인지 아닌지에 따라 카메라가 쫓아가느냐 마냐를 결정하였다. 만약 단발모드이면 플레이어의 Update함수에서 카메라에서 만든 UpdateByBullet함수를 통해 카메라를 업데이트하고, 마찬가지로 카메라가 바라보는 방향을 갱신시키기 위해 SetLookAtByBullet함수를 카메라 객체에 만들어 단발모드일 때 이처럼 카메라가 총알을 따라가면서 업데이트 되도록 구현하였다.

```

if (bullet_camera_mode) {
    DWORD nCurrentCameraMode = m_pCamera->GetMode();
    if (nCurrentCameraMode == THIRD_PERSON_CAMERA) m_pCamera->UpdateByBullet(pBulletShader->m_ppBullets[0]->GetPosition(), fTimeElapsed);
    if (m_pCameraUpdatedContext) OnCameraUpdateCallback(fTimeElapsed);
    if (nCurrentCameraMode == THIRD_PERSON_CAMERA) m_pCamera->SetLookAtByBullet(pBulletShader->m_ppBullets[0]->GetPosition());
    m_pCamera->RegenerateViewMatrix();
}
else if (!bullet_camera_mode) {
    DWORD nCurrentCameraMode = m_pCamera->GetMode();
    if (nCurrentCameraMode == THIRD_PERSON_CAMERA || nCurrentCameraMode == LEFT_CAMERA) m_pCamera->Update(m_xmf3Position, fTimeElapsed);
    if (m_pCameraUpdatedContext) OnCameraUpdateCallback(fTimeElapsed);
    if (nCurrentCameraMode == THIRD_PERSON_CAMERA || nCurrentCameraMode == LEFT_CAMERA) m_pCamera->SetLookAt(m_xmf3Position);
    m_pCamera->RegenerateViewMatrix();
}

```

6. Bullet



미사일 모델은 유니티 엔진에서 추출하려 했을 때 메시가 여러 개로 이루어져있고, 각 꼬리부분이 각도가 안 맞아 직접 수정해주었다. 또한 미사일이 기본적으로 회전하면서 날라갈 수 있도록 각 메시의 축과 상위 오브젝트의 축을 제대로 된 방향으로 일치시켜주었다. 미사일의 텍스처는 Albedo, Normal, Occlusion으로 하여 노멀 매핑을 수행하여 조명을 받을 수 있도록 Bullet을 위한 PS도 따로 작성해주었다.

미사일을 가지는 객체는 적 탱크 객체들과 플레이어이다. 적 탱크는 각자 하나씩 가지게 되고, 플레이어는 동시에 10개를 소지하고 있다. 또한, 플레이어의 총알 객체는 더 많은 처리 등을 하고, 카메라 움직임이 따라가도록 구현도 해야했으므로 CPlayerBulletObject와 CTankObjectBullet을 따로 만들어 서로 관리하도록 하였다. 초기에 탱크의 포신에서 미사일이 날라가 자체 회전을 하면서 포신이 바라보는 방향으로 쏘려고 했는데, 포신이 바라보는 쪽으로 가기는 하는데 항상 플레이어의 룩벡터 방향을 바라보면서(정면) 날아가는 문제가 있었다. 그 문제를 해결하기 위해 포신의 업벡터와 포신의 룩벡터를 실시간으로 계산하면서 외적으로 Right벡터를 구해 변환행렬로 만들어 월드행렬에 반영시켰다.

```
void CPlayerBulletObject::UpdateLooktoPoshin()
{
    XMFL0AT3 xmf3PoshinUp = static_cast<CTankPlayer*>(m_pPlayer)->m_pPoshin->GetUp();
    XMFL0AT3 xmf3PoshinLook = static_cast<CTankPlayer*>(m_pPlayer)->m_pPoshin->GetLook();
    XMFL0AT3 xmf3PoshinRight = Vector3::CrossProduct(xmf3PoshinUp, xmf3PoshinLook, true);

    m_xmf4x4Transform._11 = xmf3PoshinRight.x; m_xmf4x4Transform._12 = xmf3PoshinRight.y; m_xmf4x4Transform._13 = xmf3PoshinRight.z;
    m_xmf4x4Transform._21 = xmf3PoshinUp.x; m_xmf4x4Transform._22 = xmf3PoshinUp.y; m_xmf4x4Transform._23 = xmf3PoshinUp.z;
    m_xmf4x4Transform._31 = xmf3PoshinLook.x; m_xmf4x4Transform._32 = xmf3PoshinLook.y; m_xmf4x4Transform._33 = xmf3PoshinLook.z;
}
```



```

if (xmf3Position.y < pTerrain->GetHeight(xmf3Position.x, xmf3Position.z) && !Collided) {
    if (!m_pPlayer->machine_mode)
        m_pPlayer->bullet_camera_mode = false; //이건 켜다 켜다 하는 bool 값
    Reset();
}
UpdateBoundingBox();
if (Collided) {
    CollideLockingTime += fElapsedTime;
    fDistance = 0.f; //움직이지 못하게 하고, Collided발동시 렌더하지 않도록 함.
    if (CollideLockingTime > 5.0f) {
        if (!m_pPlayer->machine_mode) //제거 해야할수도
            m_pPlayer->bullet_camera_mode = false;
        Reset();
    }
}
if ((m_fMovingDistance > m_fBulletEffectiveRange) || (m_fElapsedTimeAfterFire > m_fLockingTime) && !Collided) {
    if (!m_pPlayer->machine_mode)
        m_pPlayer->bullet_camera_mode = false; //머신 모드가 아니고 bulletcameramode가 아닐때 쏘면 bulletcameramode로
    Reset();
}
}

```

또한 미사일의 Update함수에는 Reset을 하는 조건이 여러가지가 있는데, 그 중 지형과 충돌하여 특정 좌표에서의 지형의 높이보다 낮게 되면 총알을 초기화 하도록 하였다. 그리고 만약 총알이 적 객체를 맞출 시 적은 스프라이트 애니메이션을 펼치기 때문에 일정 시간동안 단발모드 카메라가 고정되어 줌인 상태로 맞은 객체를 보여준다.



[총알에 맞아 단발모드 카메라가 따라간 상황]

플레이어는 미사일을 여러 개를 관리하기 때문에 플레이어 클래스 내부에 CBulletsShader포인트 객체를 하나 만들어 해당 셰이더를 만들어 관리하도록 하였다. BuildObjects을 통해 여러 개의 총알을 불러온 모델을 메쉬로 가지도록 하여 관리하는 구조로 만들었다. -> 생성자에서 셰이더 관리

```

CTankPlayer::CTankPlayer(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList, ID3D12Texture* pd3dTexture)
{
    m_pCamera = ChangeCamera(/*SPACESHIP_CAMERA*/LEFT_CAMERA, 0.0f);
    m_pShader = new CPlayerShader();
    m_pShader->CreateShader(pd3dDevice, pd3dCommandList, pd3dGraphicsRootSignature);
    //m_pShader->CreateCbvSrvDescriptorHeaps(pd3dDevice, 0, 5);

    CGameObject* pGameObject = CGameObject::LoadGeometryFromFile(pd3dDevice, pd3dCommandList, pd3dTexture);

    SetChild(pGameObject);
    //pGameObject->SetScale(0.5, 0.5, 0.5);

    //SetOOBB(18.0, 12.0f, 38.0);
    SetOOBB(9.0, 15.0f, 19.0);
    CHeightMapTerrain* pTerrain = (CHeightMapTerrain*)pContext;
    SetPosition(XMFLOAT3(pTerrain->GetWidth() * 0.5f, 255.0f, pTerrain->GetLength() * 0.5f));
    Rotate(0.f, 180.f, 0.f);
    SetPlayerUpdatedContext(pTerrain);
    SetCameraUpdatedContext(pTerrain);

    PrepareAnimate();

    m_pPlayerShader = new CBulletsShader();
    ((CBulletsShader*)m_pPlayerShader)->m_pPlayer = this;
    ((CBulletsShader*)m_pPlayerShader)->m_pCamera = m_pCamera;
    m_pPlayerShader->CreateShader(pd3dDevice, pd3dCommandList, pd3dGraphicsRootSignature);
    m_pPlayerShader->BuildObjects(pd3dDevice, pd3dCommandList, pd3dGraphicsRootSignature, pContext);
}

```

총알은 또한 반짝이는 효과를 넣기 위해 Glow인자들을 셰이더 코드의 PS에서 넣었는데, 기존의 좀 밝은 모델을 사용했을 때는 자체적으로 반짝였는데, 현재 사용하고 있는 모델 자체의 색이 어두워서 잘 효과가 나타나지는 않는 것 같다.

```

float4 cIllumination = float4(1.0f, 1.0f, 1.0f, 1.0f);
float4 cColor = cAlbedoColor + cSpecularColor + cEmissionColor;

// Calculate glow effect
float4 glow = float4(0.0f, 0.0f, 0.0f, 0.0f);
float3 glowColor = float3(0.0f, 1.0f, 1.0f);
float glowThreshold = 0.3f; // 조정 가능한 임계값
float glowIntensity = 5.0f; // 조정 가능한 강도

if (gnTexturesMask & MATERIAL_NORMAL_MAP)
{
    float3 normalW = input.normalW;
    float3x3 TBN = float3x3(normalize(input.tangentW), normalize(input.bitangentW), normalize(input.normalW));
    float3 vNormal = normalize(cNormalColor.rgb * 2.0f - 1.0f); // [0, 1] → [-1, 1]
    normalW = normalize(mul(vNormal, TBN));
    cIllumination = Lighting(input.positionW, normalW);

    // Calculate the glow based on the illumination
    if (cIllumination.r > glowThreshold)
    {
        glow.r = (cIllumination.r - glowThreshold) * glowIntensity;
    }

    if (cIllumination.g > glowThreshold)
    {
        glow.g = (cIllumination.g - glowThreshold) * glowIntensity;
    }

    if (cIllumination.b > glowThreshold)
    {
        glow.b = (cIllumination.b - glowThreshold) * glowIntensity;
    }

    // Apply the glow to the color
    cColor += glow;
}

```

[PSBullet- glow 효과]

```

CPlayerBulletObject** ppBullets = ((CBulletsShader*)((CTankPlayer*)m_pPlayer)->m_pPlayerShader)->m_ppBullets;
CTankObjectsShader* pEnemyShader = (CTankObjectsShader*)m_ppShaders[ENEMYTANK_INDEX];
for (int i = 0; i < pEnemyShader->m_nTanks; i++) {
    for (int j = 0; j < BULLETS; j++) {
        if (ppBullets[j]->m_bActive && pEnemyShader->m_ppTankObjects[i]->m_xmCollision.Intersects(ppBullets[j]->m_xmCollision))
        {
            //ppBullets[j]->Collided = true;
            if (!static_cast<CTankObject*>(pEnemyShader->m_ppTankObjects[i])->hitByBullet) {
                //소리 on
                static_cast<CTankObject*>(pEnemyShader->m_ppTankObjects[i])->hitByBullet = true; //탱크 맞으면 - 탱크 관련 처리
                ppBullets[j]->Collided = true; //총알 맞으면 - 총알 관련 처리
                for (int k = 0; k < m_nSpriteAnimation; k++) {
                    m_ppSprite[k]->m_bActive = true; // 탱크 맞으면 스프라이트 ON
                }
            }

            if (static_cast<CTankObject*>(pEnemyShader->m_ppTankObjects[i])->hitByBullet) {
                Game_Sound.Sound_ON = true;
                XMFLQAT3 xmf3CameraPosition = m_pPlayer->GetCamera()->GetPosition();
                CPlayer* pPlayer = m_pPlayer;
                XMFLQAT3 xmf3PlayerPosition = pPlayer->GetPosition();
                XMFLQAT3 xmf3PlayerLook = pPlayer->GetLookVector();
                XMFLQAT3 xmf3EnemyPosition = pEnemyShader->m_ppTankObjects[i]->GetPosition();
                XMFLQAT3 GetPosDifference = Vector3::Subtract(xmf3EnemyPosition, xmf3PlayerPosition, false);
                float GetLength = Vector3::Length(GetPosDifference);
                xmf3EnemyPosition.y += 5.0f;
                XMFLQAT3 xmf3Position = Vector3::Add(xmf3PlayerPosition, Vector3::ScalarProduct(xmf3PlayerLook, GetLength, false));
                for (int k = 0; k < m_nSpriteAnimation; k++) {
                    m_ppSprite[k]->SetPosition(xmf3EnemyPosition);
                    m_ppSprite[k]->SetLookAt(xmf3CameraPosition, XMFLQAT3(0.0f, 1.0f, 0.0f));
                }
            }
        }
    }
}

```

총알이 적 탱크와 충돌처리를 함과 동시에 만약 맞았을 때 스프라이트 애니메이션을 활성화 시키는 코드이다. 또한 총알에 맞은 적이 있다면 Sound도 활성화하여 폭발 브금이 커진다.

7. Billboard

빌보드는 조준점을 구현하기 위해 만들었다. 조준점은 총 3가지 모드가 있으며, 모드마다 사용하는 빌보드의 개수가 다르다. 빌보드는 조준점 목적으로 만들었기 때문에 빌보드는 Poshin을 바라보도록 하고 BillboardObject의 Animate함수에서 distance 인자를 받아 포신의 위치로부터 얼마나 룩벡터 방향으로 거리를 두면서 위치를 잡을지 Animate하면서 계산하면서 이동하게끔 구현하였다. Billboard는 씬에서 관리하도록 구현하였다. 수량이 많지 않아 셰이더로 따로 관리하지는 않고 하나씩 만들어서 씬에서 관리하도록 하였지만, 대신 씬에서 넘겨주기 위해 생성자에 FilePath, width, height를 건내주도록 바뀌었다.

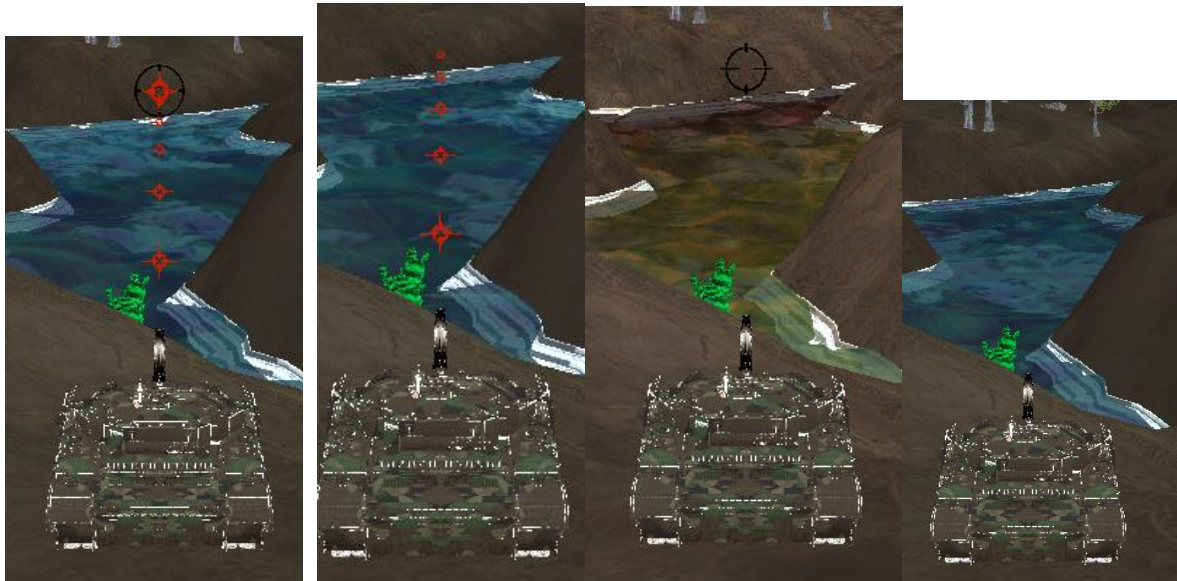
```
m_ppDotBillboard[5] = new CBillboardObject(pd3dDevice, pd3dCommandList, m_pd3dGraphicsRootSignature, L"Image/Dot1.dds", 10.0f, 10.0f);  
m_ppDotBillboard[6] = new CBillboardObject(pd3dDevice, pd3dCommandList, m_pd3dGraphicsRootSignature, L"Image/Dot2.dds", 10.0f, 10.0f);  
m_ppDotBillboard[7] = new CBillboardObject(pd3dDevice, pd3dCommandList, m_pd3dGraphicsRootSignature, L"Image/Dot_Sight.dds", 10.0f, 10.0f);
```

```
void CBillboardObject::Animate(float fTimeElapsed, CCamera* pCamera, float distance)  
{  
    XMFLQAT3 xmf3CameraPosition = pCamera->GetPosition();  
    CPlayer* pPlayer = pCamera->GetPlayer();  
    XMFLQAT3 xmf3PlayerPosition = static_cast<CTankPlayer*>(pPlayer)->m_pPoshin->GetPosition();  
    XMFLQAT3 xmf3PlayerLook = static_cast<CTankPlayer*>(pPlayer)->m_pPoshin->GetLook();  
  
    XMFLQAT3 xmf3Position = Vector3::Add(xmf3PlayerPosition, Vector3::ScalarProduct(xmf3PlayerLook, distance, false));  
  
    SetPosition(xmf3Position);  
    SetLookAt(xmf3CameraPosition);  
}  
  
void CBillboardObject::SetLookAt(XMFLQAT3& xmf3Target)  
{  
    XMFLQAT3 xmf3Up(0.f, 1.f, 0.f);  
    XMFLQAT3 xmf3Position(m_xmf4x4Transform._41, m_xmf4x4Transform._42, m_xmf4x4Transform._43);  
    XMFLQAT3 xmf3Look = Vector3::Subtract(xmf3Target, xmf3Position, true);  
    XMFLQAT3 xmf3Right = Vector3::CrossProduct(xmf3Up, xmf3Look, true);  
    m_xmf4x4Transform._11 = xmf3Right.x; m_xmf4x4Transform._12 = xmf3Right.y; m_xmf4x4Transform._13 = xmf3Right.z;  
    m_xmf4x4Transform._21 = xmf3Up.x; m_xmf4x4Transform._22 = xmf3Up.y; m_xmf4x4Transform._23 = xmf3Up.z;  
    m_xmf4x4Transform._31 = xmf3Look.x; m_xmf4x4Transform._32 = xmf3Look.y; m_xmf4x4Transform._33 = xmf3Look.z;  
}
```

빌보드에서 또 중요한건, 빌보드에 매핑한 텍스처는 알파값 정보를 잘 보존하고 있다. 그를 투명하게 표현하기 위해서는 Blending을 해줘야한다 그렇기 때문에 SrcBlend에 SrcAlpha를 넣어주고 DestBlend에는 INV_SRC_ALPHA를 넣어주어 알파 블렌딩을 해주어 투명한 부분을 처리해주고 AlphatoCoverage를 켜 경계선 부분에 대한 멀티샘플링을 적용시켜 부드러운 경계선을 표현하도록 하였다.

또한, 항상 앞을 바라보도록 할 것이기 때문에 래스터라이저 스테이트도 바꿔주어야한다. Cull모드를 후면 면을 제거하게끔 MODE_BACK으로 하고, 시계방향을 기준으로 렌더링

하고, DepthClip을 허용하게 하여 빌보드에 맞는 렌더링 설정을 해주고 그리게 하였다.



빌보드는 기본적으로 TextureRectMesh로 만들기 때문에 InputLayout이 pos, texcoord 두 개 정보만 정점이 들고 있으면 된다. 그렇게 받은 정보를 W,P,V행렬들을 각각 곱하고 텍스처로부터 읽어온 텍스처를 각 uv에 맞게 매핑하면 빌보드는 잘 만들어진다.

8. Water

[Terrain RectMesh Water] - 물을 표현하기 위해 샘플 프로젝트에서 빌보드와 같이 TextureRectMesh로 지형의 크기만큼 만들어 x,z축으로 늘린 형태로 하여 텍스처를 입히고, 그 텍스처의 uv값을 PS에서 움직이는 형태로 만든 TerrainWater부터 추가하였다. 기존 프로젝트에 TextureRectMesh는 정점을 Pos는 XMFLOAT3로 정의하였고 TexCoord는 XMFLOAT2로 하여 정점버퍼를 두개 만들어 Mesh를 그릴 때 IA에 정점버퍼를 두가지 형태를 Set하도록 되어 있었다. 이 형태를 바꾸기 위해 새로운 메쉬를 만들어 TextureRectMeshWithOneVertex를 만들어 새로운 정점형태를 설명하는 class를 만들고 정점 xmf3Position과 xmf2Texcoord를 가지도록 상속하였다. 이렇게 만들어 하나의 정점 버퍼로 그릴 수 있도록 하였다.



그 결과 평평한 물이 uv값들이 바뀌면서 움직이는 형태로 완성되었다.

```
pd3dRootParameters[12].ParameterType = D3D12_ROOT_PARAMETER_TYPE_32BIT_CONSTANTS; //비용 2
pd3dRootParameters[12].Constants.Num32BitValues = 2; //Time, ElapsedTime
pd3dRootParameters[12].Constants.ShaderRegister = 3; //Time
pd3dRootParameters[12].Constants.RegisterSpace = 0;
pd3dRootParameters[12].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

pd3dRootParameters[13].ParameterType = D3D12_ROOT_PARAMETER_TYPE_CBV; //비용 2 루트 상수에서 CBV로 바꿈.
pd3dRootParameters[13].Constants.ShaderRegister = 4; //WaterTextureAnimation 행렬
pd3dRootParameters[13].Constants.RegisterSpace = 0;
pd3dRootParameters[13].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

pd3dRootParameters[14].ParameterType = D3D12_ROOT_PARAMETER_TYPE_DESCRIPTOR_TABLE; //비용 1
pd3dRootParameters[14].DescriptorTable.NumDescriptorRanges = 1;
pd3dRootParameters[14].DescriptorTable.pDescriptorRanges = &pd3dDescriptorRanges[9]; //Water 관련 텍스처 3개
pd3dRootParameters[14].ShaderVisibility = D3D12_SHADER_VISIBILITY_PIXEL;
```

이를 구현하기 위해 루트시그니처에 추가해야하는 요소가 세가지 있었다.

첫번째는 애니메이션이 동작하게 하기 위해 GameTimer의 현재시간과 GameTimer의 경과된 시간들을 GameFramework.cpp에서 UpdateShaderVariables함수를 추가하였다. 이 UpdateShaderVariables함수는 FrameAdvance함수에서 커맨드리스트가 Reset된 이후

Scene을 Render하기 전에 호출하도록 하였다. 또한 그러기 위해 미리 씬의 루트시그니처를 바인딩해야만 타이머에 대한 루트 상수들이 Update 하였을 때 셰이더에 제대로 반영되므로 그전에 Scene에서 SetGraphicsRootSignature을 호출하도록하는 PrepareRender을 추가하였다.

두번째는 물이 애니메이션처럼 보이게 하기 위한 행렬이 필요했다. 이는 상수버퍼 뷰 형태로 건내주도록 하였고, 행렬 자체는 씬에서 관리하도록 하였다. 씬의 생성자에서 이 행렬을 기본 Identity행렬로 초기화하도록 하고, 씬에서 CreateShaderVariables에서 버퍼를 만들어 리소스 포인터에 바인딩하여 데이터를 업로드할 수 있도록 만들었다. 씬의 UpdateShaderVariables에는 조명관련 인자들을 상수버퍼뷰에 업데이트 하고 있는데, 여기에 행렬을 Transpose하여 상수버퍼뷰와 매핑된 행렬 포인터에 Store하여 업데이트 하도록 하였다. 그리고 Animate을 구현하기 위해 CScene의 AnimateObjects함수에서 2D객체 기준 이동변환에 해당하는 인자를 바꾸도록 구현하였다.

세번째는 물을 표현하기 위한 텍스처를 사용하기 위한 셰이더리소스뷰들을 셰이더에 건내주어야 했다. 그러기 위해 디스크립터 힙을 셰이더에서 만들고 셰이더에서 사용하는 레지스터를 사용하는 루트시그니처 인덱스에 맞는 셰이더 리소스 뷰들을 만들어 세팅을 해주었다.

[Ripple Water] – 물이 평평하다 보니 출렁거리는 형태의 물을 구현하고 싶었다. 그러기 위해 샘플 프로젝트에 있는 RippleWater를 나의 프로젝트에 추가하였다. TerrainWater를 구현했을 때 사용한 행렬과 Time관련 데이터를 마찬가지로 셰이더에서 사용하기 때문에 오브젝트 구조와 셰이더 코드를 바꾸면 되는 작업이었다. Ripple Water은 기본적으로 지형을 만들었을 때 처럼 격자모양의 메쉬를 사용하였는데, 그렇기 때문에 지형과 격자와 Scale과 가로 세로 크기를 일치 시켜주어야 했다. 메쉬는 GridMesh형태로 Set을 하게 되었는데, 나의 프로젝트에 병합할 때 바꾸어야하는 요소들이 있었다. 샘플 프로젝트에는 기본적으로 Terrain과 RippleWater이 같은 생성자 하에 만들어지도록 하고 HeightImage의 여부에 따라 설정되는 값만 살짝 달랐다. 나는 Terrain은 정점을 이루는 구조가 Position, Normal, Texcoord두개로 이루어져 있어 이 형태로 만들 수 없었다. 그렇기 때문에 새로운 정점을 설명하는 CNormalTexturedVertexOne을 만들어 RippleWater에서의 메쉬인 GridMesh를 구성할 때 사용하였다. RippleWater도 조명효과를 받을 수 있도록 Normal값을 주도록 하였다. 하지만 생기는 문제는 Terrain은 HeightMapImage의 데이터를 통해 구하기 때문에 HeightMap에 있는 GetNormal을 통해 Normal값을 유추할 수 있었지만, RippleWater는 HeightMapImage를 사용하지 않기 때문에 이와 같은 방법은 사용하지 못했다. 그렇기 때문에 지형을 새로 만들어 물을 표현하는 방법도 고려해보았는데,

일단은 지형을 만들 때 사용한 HeightMapImage를 생성자의 Context로 받아와 Normal 값을 일치 시켜주었다.

```
// CTexturedVertex *pVertices = new CTexturedVertex[m_nVertices];
CNormalTexturedVertexOne* pVertices = new CNormalTexturedVertexOne[m_nVertices];

//CHightMapImage* pHeightMapImage = (CHightMapImage*)pContext;
int cxHeightMap = m_nWidth;
int czHeightMap = m_nLength;
/*if (pHeightMapImage)
{
    cxHeightMap = pHeightMapImage->GetHeightMapWidth();
    czHeightMap = pHeightMapImage->GetHeightMapLength();
}*/

float fHeight = 0.0f, fMinHeight = +FLT_MAX, fMaxHeight = -FLT_MAX;
for (int i = 0, z = zStart; z < (zStart + nLength); z++)
{
    for (int x = xStart; x < (xStart + nWidth); x++, i++)
    {
        fHeight = 0nGetHeight(x, z, pContext);
        pVertices[i].m_xmf3Position = XMFLOAT3((x * m_xmf3Scale.x), fHeight, (z * m_xmf3Scale.z));
        pVertices[i].m_xmf3Normal = static_cast<CHightMapTerrain*>(pContext)->GetNormal((x * m_xmf3Scale.x), fHeight, (z * m_xmf3Scale.z));

        pVertices[i].m_xmf2TexCoord = XMFLOAT2(float(x) / float(cxHeightMap - 1), float(czHeightMap - 1));
        if (fHeight < fMinHeight) fMinHeight = fHeight;
        if (fHeight > fMaxHeight) fMaxHeight = fHeight;
    }
}
```

```
CTexture* pWaterTexture = new CTexture(3, RESOURCE_TEXTURE2D, 0, 1);
pWaterTexture->LoadTextureFromDDSFile(pd3dDevice, pd3dCommandList, L"Image/My_Water_Base.dds", RESOURCE_TEXTURE2D, 0);
pWaterTexture->LoadTextureFromDDSFile(pd3dDevice, pd3dCommandList, L"Image/Water_Pool.dds", RESOURCE_TEXTURE2D, 1);
pWaterTexture->LoadTextureFromDDSFile(pd3dDevice, pd3dCommandList, L"Image/Detailed.dds", RESOURCE_TEXTURE2D, 2);
```

텍스처는 새로 인터넷에서 물을 잘 표현한 것 같은 텍스처 두개와 흰색 무늬 형태로 물의 거품들이나 이물질을 표현할 수 있는 Detail 텍스처 하나를 dds파일로 변환하여 불러오도록 바꾸었다.

마지막으로 RippleWater은 높이를 180.0f로 설정하도록 썬과 셰이더에서 변경하였다.

```
// input.position.y += sin(gfCurrentTime * 0.35f + (input.position.x * input.position.x) + (input.position.z * input.position.z));
input.position.y += sin(gfCurrentTime * 0.35f + input.position.x * 0.35f) * 5.95f + cos(gfCurrentTime * 0.30f + input.position.z * 0.30f) * 5.95f;
output.positionW = (float3)mul(float4(input.position, 1.0f), gmtxGameObject);
output.position = mul(float4(input.position, 1.0f), gmtxGameObject);
if (180.0f < output.position.y) output.position.y = 180.0f;
output.position = mul(mul(output.position, gmtxView), gmtxProjection);

output.normalW = mul(input.normal, (float3x3)gmtxGameObject);
```

```
input.normalW = normalize(input.normalW);
float4 cIllumination = Lighting(input.positionW, input.normalW);

cColor = lerp(cColor, cIllumination, 0.3f);
return cColor;
```

조명과 Position을 180으로 정하기 위해 셰이더 코드 변경.



→ 샘플 프로젝트 텍스처로 띄웠을 때



→ 나의 텍스처로 바꾸고 조명을 적용하였을 때.

RippleWater 형태로 씬에서 받으면 탱크가 물에 들어가면 물에 뜨고 물에 뜨는 애니메이션을 하도록 구현하였다. 물에 들어가면 상하로 왔다갔다하고 지형에 가면 다시 정상적으로 동작한다. 물에 들어가면 중력이 0이 된다.



[물에 뜬 탱크 플레이어의 모습]

현재는 추가적으로 물에 대한 반투명을 표현하기 위해 CreateBlendState 가상 함수를 정의하여 CSrc(원본혼합 계수)를 SRC의 알파값, Cdst(대상 혼합 계수)를 SRC의 알파값의 inverse로 정하고 BLEND_OP_ADD로 연산하도록 하였다. 또한, 그리는 순서를 불투명 물물부터 그려 카메라로부터 거리가 먼 물체들과 반투명한 물체의 색상이 혼합되도록 하였다.

```
UpdateShaderVariables();
m_pScene->Render(m_pd3dCommandList, m_pCamera);

#ifdef _WITH_PLAYER_TOP
    m_pd3dCommandList->ClearDepthStencilView(d3dDsvCPUDescriptorHandle, D3D12_CLEAR_FLAGS_DEPTH_STENCIL);
#endif
    if (m_pPlayer) m_pPlayer->Render(m_pd3dCommandList, m_pCamera);
    m_pScene->RenderTransparentAfterPlayer(m_pd3dCommandList, m_pCamera);
#ifdef _WITH_DIRECT2D
```

```

D3D12_BLEND_DESC CrippleWaterShader::CreateBlendState()
{
    D3D12_BLEND_DESC d3dBleNDDesc;
    ::ZeroMemory(&d3dBleNDDesc, sizeof(D3D12_BLEND_DESC));
    d3dBleNDDesc.AlphaToCoverageEnable = TRUE;
    d3dBleNDDesc.IndependentBlendEnable = FALSE;
    d3dBleNDDesc.RenderTarget[0].BlendEnable = TRUE;
    d3dBleNDDesc.RenderTarget[0].LogicOpEnable = FALSE;
    d3dBleNDDesc.RenderTarget[0].SrcBlend = D3D12_BLEND_SRC_ALPHA;
    d3dBleNDDesc.RenderTarget[0].DestBlend = D3D12_BLEND_INV_SRC_ALPHA;
    d3dBleNDDesc.RenderTarget[0].BlendOp = D3D12_BLEND_OP_ADD;
    d3dBleNDDesc.RenderTarget[0].SrcBlendAlpha = D3D12_BLEND_ONE;
    d3dBleNDDesc.RenderTarget[0].DestBlendAlpha = D3D12_BLEND_ZERO;
    d3dBleNDDesc.RenderTarget[0].BlendOpAlpha = D3D12_BLEND_OP_ADD;
    d3dBleNDDesc.RenderTarget[0].LogicOp = D3D12_LOGIC_OP_NOOP;
    d3dBleNDDesc.RenderTarget[0].RenderTargetWriteMask = D3D12_COLOR_WRITE_ENABLE_ALL;

    return(d3dBleNDDesc);
}

```

[물을 반투명으로 만들기 위해 알파블렌딩을 블렌드 스테이트에 적용한 것]



-> 탱크가 반투명한 물 아래에서도 보이는 모습 (불투명 물체를 먼저 그린 결과)

물이 탱크 플레이어의 색상과 혼합되기 위해서는 무조건 불투명한 물체들부터 그려야 하므로 씬에서 관리하고 있던 RippleWater를 따로 렌더하는 함수를 만들어 GameFrameWork에서 씬관련 렌더링을 수행하고 플레이어를 렌더링하고 RippleWater만 따로 렌더하는 함수를 실행하여 렌더하는 순서를 지켰다.

9. Tree

나무는 모델을 두가지 준비하였다. 단일 나무 모델과, 여러 나무들을(4개)를 묶은 모델을 구현했다. 유니티에서 직접 단일 나무를 위치시켜 하나의 모델처럼 추출하였다. 나무와 같이 맵을 꾸미는 자원들은 유니티에서 원점을 기준으로 내가 비치한 상태를 묶어서 추출하면 그대로 맵에 위치하면 엔진에서 꾸민 효과를 실제 모델을 렌더했을 때 그대로 볼 수 있다. 처음에는 숲을 만들기 위해 많은 나무들을 묶어서 추출하려 했는데, 나무가 모델이 용량도 크고 많은 폴리곤으로 이루어져 있어 많이 렌더하면 프레임 레이트가 떨어지는 현상이 있었다. 나무는 알파블렌딩과 알파to커버리지를 활성화해야만 잎사귀 부분이 자연스럽게 나뭇잎 사이사이에 투명한 부분이 보인다. 나무는 노멀맵 등 다양한 텍스처가 있지만 PixelShader에서 노멀맵과 알bedo 텍스처만 적용하였다. 나무는 일반적으로 지형에서 물이 있는 곳을 제외한 랜덤한 곳에 위치를 시켰고 묶음 형태의 나무는 10개, 일반 단일 나무는 30개를 렌더하였다. 나무는 빌보드와 마찬가지로 은면에 대해서 컬링하고 RasterizeState를 재정의 해야하고, DepthStencilState도 적용해서 깊이값을 누적으로 쓰면서 depth값이 작거나 같은 기준으로 깊이 검사를 하도록 COMPARISION_FUNC_LESS_EQUAL로 하였다.(거의 기본 설정과 같다), 스텐실은 활성화 하지 않았다.



[유니티에서 추출한 나무 묶음 모델]



```
float4 PSTree(VS_STANDARD_OUTPUT input) : SV_TARGET
{
    float4 cAlbedoColor = float4(0.0f, 0.0f, 0.0f, 1.0f);
    float4 cNormalColor = float4(0.0f, 0.0f, 0.0f, 1.0f);

#ifdef _WITH_STANDARD_TEXTURE_MULTIPLE_DESCRIPTOR
    if (gnTexturesMask & MATERIAL_ALBEDO_MAP) cAlbedoColor = gtxtAlbedoTexture.Sample(gssWrap, input.uv);
    if (gnTexturesMask & MATERIAL_NORMAL_MAP) cNormalColor = gtxtNormalTexture.Sample(gssWrap, input.uv);
#else
    if (gnTexturesMask & MATERIAL_ALBEDO_MAP) cAlbedoColor = gtxtStandardTextures[0].Sample(gssWrap, input.uv);
    if (gnTexturesMask & MATERIAL_NORMAL_MAP) cNormalColor = gtxtStandardTextures[1].Sample(gssWrap, input.uv);
#endif

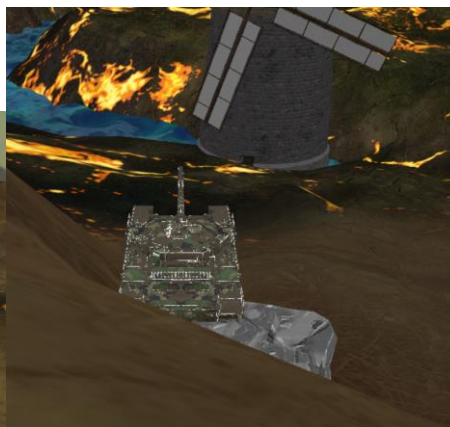
    float4 cIllumination = float4(1.0f, 1.0f, 1.0f, 1.0f);
    float4 cColor = cAlbedoColor;
    if (gnTexturesMask & MATERIAL_NORMAL_MAP)
    {
        float3 normalW = input.normalW;
        float3x3 TBN = float3x3(normalize(input.tangentW), normalize(input.bitangentW), normalize(input.normalW));
        float3 vNormal = normalize(cNormalColor.rgb * 2.0f - 1.0f); // [0, 1] → [-1, 1]
        normalW = normalize(mul(vNormal, TBN));
        cIllumination = Lighting(input.positionW, normalW);
        cColor = lerp(cColor, cIllumination, 0.5f);
    }
}
```

10. Rocks And Cactus

일단 맵을 꾸미기 위해 텍스처는 Albedo 텍스처 하나만 사용하고 비용이 덜 드는 오브젝트를 찾아보았다. 들판 느낌에서 나무도 있고 바닥은 풀, 잡초로 이루어진 곳이라 식물이나 돌이 있으면 좋을 것 같다는 생각을 하였다. 그래서 잡초처럼 생긴 선인장 모델과 울퉁불퉁한 돌 모델을 유니티에서 추출하였다. 초기에 각 모델을 추출할 때 1by 1크기의 큐브를 이용하여 각 모델의 크기를 상대적으로 비교하였다. 그 비교한 결과를 가지고 바운딩박스 크기를 정해주었다. 이렇게 바운딩 박스를 적용한 이유는 선인장은 탱크 플레이어 밟고 지나가면 특정 흔들리는 애니메이션과 함께 선인장이 납작하게 밟힌 모습을 보여주려고 하였고, 돌은 충돌처리를 적용하여 각 면마다 탱크 플레이어가 부딪힐 수 있고 지속적으로 해당 방향으로 밀면 돌이 밀리게끔 구현하였다. 돌과 선인장의 충돌은 기본적으로 플레이어만 충돌할 수 있도록 구현해놓았다.



[선인장 밟고 지나갔을 때 탱크가 0.4초동안 좌우로 흔들리고 선인장은 밟힘]



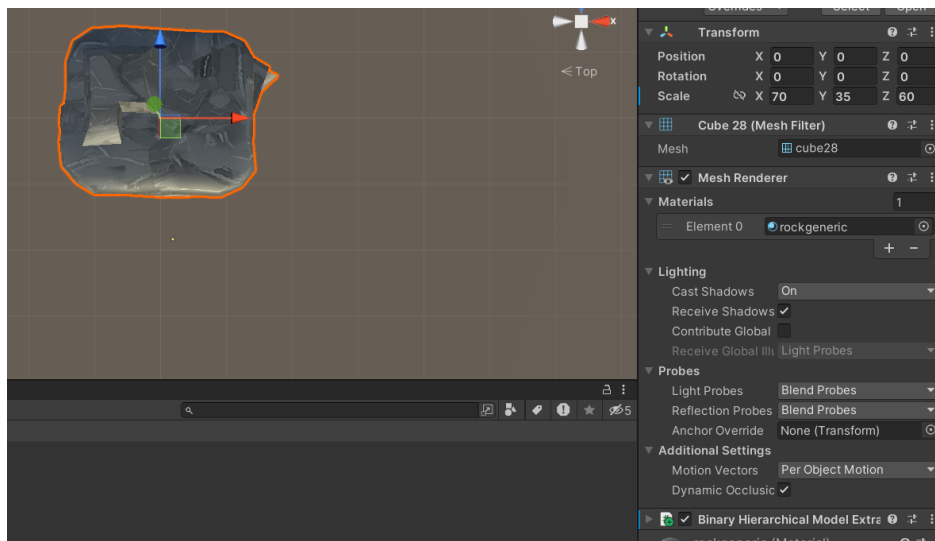
[돌을 밀어서 옮긴 모습]

```
void CScene::CheckRockByTankCollisions(float fTimeElapsed)
{
    CactusAndRocksShader* pShader = static_cast<CactusAndRocksShader*>(_m_ppShaders[CACTUS_AND_ROCKS_INDEX]);
    XMFL0AT3 pos = _m_pPlayer->GetPosition();

    for (int i = 0; i < pShader->m_nRock; i++) {
        XMFL0AT3 Rockpos = pShader->m_ppRocks[i]->GetPosition();

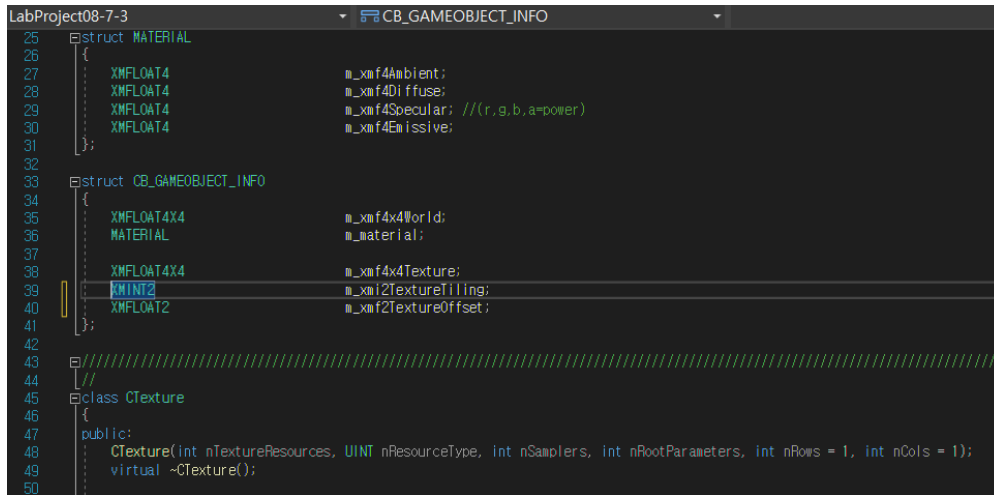
        if (_m_pPlayer->GetPosition().z - 4.f < pShader->m_ppRocks[i]->GetPosition().z + 11.f && _m_pPlayer->GetPosition().z + 4.f > pShader->m_ppRocks[i]->GetPosition().z) {
            if (_m_pPlayer->GetPosition().x - 9.f < pShader->m_ppRocks[i]->GetPosition().x + 13.f && _m_pPlayer->GetPosition().x + 9.f > pShader->m_ppRocks[i]->GetPosition().x) {
                if (_m_pPlayer->GetPosition().x > pShader->m_ppRocks[i]->GetPosition().x) {
                    pos.x = pShader->m_ppRocks[i]->GetPosition().x + 22.f;
                    _m_pPlayer->SetPosition(pos);
                    pShader->m_ppRocks[i]->SetPosition(Rockpos.x - 5.0f * fTimeElapsed, Rockpos.y, Rockpos.z);
                    XMFL0AT3 iasipos = pShader->m_ppRocks[i]->GetPosition();
                    iasipos.y = _m_pTerrain->GetHeight(iasipos.x, iasipos.z);
                    pShader->m_ppRocks[i]->SetPosition(iasipos.x, iasipos.y, iasipos.z);
                }
                else {
                    pos.x = pShader->m_ppRocks[i]->GetPosition().x - 22.f;
                    _m_pPlayer->SetPosition(pos);
                    pShader->m_ppRocks[i]->SetPosition(Rockpos.x + 5.0f * fTimeElapsed, Rockpos.y, Rockpos.z);
                    XMFL0AT3 iasipos = pShader->m_ppRocks[i]->GetPosition();
                    iasipos.y = _m_pTerrain->GetHeight(iasipos.x, iasipos.z);
                    pShader->m_ppRocks[i]->SetPosition(iasipos.x, iasipos.y, iasipos.z);
                }
            }
        }
        else if (pShader->m_ppRocks[i]->m_xmCollision.Intersects(_m_pPlayer->m_xmCollision)) {
            if (_m_pPlayer->GetPosition().x - 4.f < pShader->m_ppRocks[i]->GetPosition().x + 13.f && _m_pPlayer->GetPosition().x + 4.f > pShader->m_ppRocks[i]->GetPosition().x) {
                if (_m_pPlayer->GetPosition().z - 9.f < pShader->m_ppRocks[i]->GetPosition().z + 11.f && _m_pPlayer->GetPosition().z + 9.f > pShader->m_ppRocks[i]->GetPosition().z) {
                    if (_m_pPlayer->GetPosition().z > pShader->m_ppRocks[i]->GetPosition().z) {
                        pos.z = pShader->m_ppRocks[i]->GetPosition().z + 20.f;
                        _m_pPlayer->SetPosition(pos);
                        pShader->m_ppRocks[i]->SetPosition(Rockpos.x, Rockpos.y, Rockpos.z - 5.0f * fTimeElapsed);
                    }
                }
            }
        }
    }
}
```

[돌 충돌 및 밀어내는 함수]

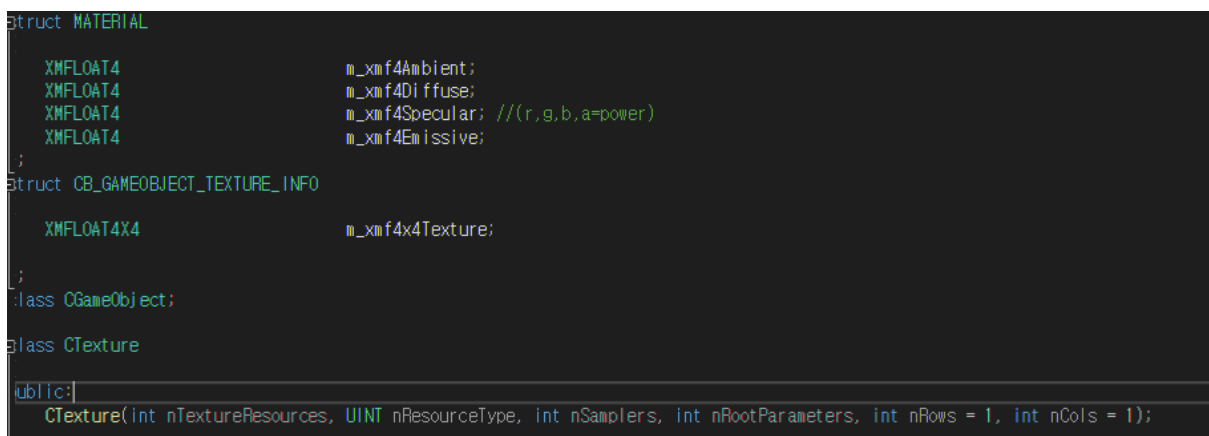


[돌을 큐브와 비교하여 크기를 가늠하여 충돌범위를 측정]

11. Sprite Animation



[LabProject예시]



우선 스프라이트 애니메이션을 적용하기 위해 CTexture에 없는 nRows와 nCols를 추가하였다. 또한 CTexture내부에 스프라이트 애니메이션이 동작하도록 하기 위한 행렬을 XMFLOAT4X4형태로 하나 추가하였다. 스프라이트 애니메이션을 기존 나의 코드에 병합할 때 가장 문제가 되는 것이 상수버퍼를 어떻게 구성하는지에 대한 것이었다. 샘플 프로젝트에서는 GameObject의 월드 행렬, 재질 정보의 구조체, 스프라이트 애니메이션을 위한 행렬과 다른 Offset인자들을 묶어서 오브젝트 클래스에서 관리하도록 되어있었다. 하지만 나의 프로젝트에서는 CMaterial에서 재질정보를 루트상수로 넘겨주도록 관리되어 있고, 월드행렬은 CGameObject에서 루트상수로 넘겨주도록 되어있었다. 그렇기 때문에 다시 상수버퍼 뷰로 재정의하기에는 현재 추가한 오브젝트들이 너무 많아 상수버퍼뷰를 관리하기 위한 추가 코드가 너무 많이 추가해야하는 비용이 있었다. 그렇기 때문에 샘플 프로젝트를 유심히 관찰한 결과 Offset과 같은 인자들은 현재 예시에서 사용하지 않기 때문에 m_xmf4x4Texture만 적절히 셰이더로 넘겨주기만 하면 될 것 같았다. 그렇기 때문

에 기존에 CGameObject에서는 모두 루트 상수로 관리하고 있기 때문에, CreateShaderVariables등이 비어있어 여기서 관리하면 될 것 같았다. 하지만 또 다른 문제는 다른 오브젝트들은 Texture들을 모두 이용은 하지만 저 Texture 애니메이션 관련 행렬을 사용하지 않고 그대로 Identity행렬을 넘겨준다는 것이었다. 비용낭비가 일어나는 것 같아 따로 CMulitSpriteObject클래스에서 CreateShaderVariables, UpdateShaderVariables, ReleaseShaderVariables에서 상수버퍼뷰를 관리하도록 하였다.

➔ [문제점] 처음에 샘플 프로젝트 처럼 SpriteAnimationShader에서 CMulitSpriteObject두개를 만들어 관리하려고 시도하였다. 하지만 샘플 프로젝트와 나의 프로젝트의 가장 큰 차이점이 CScene에서 디스크립터 힙을 관리하냐 CShader에서 관리하냐의 차이가 있었다. 그렇기 때문에 BuildObjects에서 구성 방식이나 순서가 살짝 다르게 구성하였는데, 다 만들고 보니 동작은 하는데 렌더링이 되지 않는 문제가 있었다. 아마 알파 스텐실을 적용한 스프라이트 애니메이션이기 때문에 불투명물체들을 먼저 그리고 스프라이트 객체들을 그리는게 중요한데 여기서 실수 했거나 상수버퍼뷰를 자원의 수만큼 한번에 셰이더에서 바인딩 하거나 해야 동작했을 것 같다.

➔ [대응책] 일단 동작하게 하기 위해 다른 방법을 선택하였다. 씬에서 CmulitSpriteObject 두개를 관리하게 하고 CmulitSpriteObject 생성자 내부에서 셰이더 만들고 디스크립터 힙 만들고 텍스처 만들게 하였다. 하지만 다르게 해줘야하는 부분은 씬에서 객체를 만드므로 각 객체마다 로드하는 텍스처도 다르고 새로 추가한 nRows, nCols를 다르게 해주어야하기 때문에 생성자 인자에서 받게끔 구성하였다. 시간이 남으면 셰이더에서 관리하도록 변경할 것이다.

```
void CScene::BuildObjects(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList)
{
    m_pd3dGraphicsRootSignature = CreateGraphicsRootSignature(pd3dDevice);

    BuildDefaultLightsAndMaterials();

    //m_pBillboard = new CBillboardObject(pd3dDevice, pd3dCommandList, m_pd3dGraphicsRootSignature);
    //빌보드 위치 초기화
    m_nSpriteAnimation = 2;
    m_ppSprite = new CMultiSpriteObject * [m_nSpriteAnimation];
    m_ppSprite[0] = new CMultiSpriteObject(pd3dDevice, pd3dCommandList, m_pd3dGraphicsRootSignature, L"Image/Explode_8x8.dds", 8, 8);
    m_ppSprite[1] = new CMultiSpriteObject(pd3dDevice, pd3dCommandList, m_pd3dGraphicsRootSignature, L"Image/Explosion_6x6.dds", 6, 6);
}
```

-> 씬에서 BuildObjects에서 만들도록 구성.

```

struct MATERIAL
{
    float4      m_cAmbient;
    float4      m_cDiffuse;
    float4      m_cSpecular; //a = power
    float4      m_cEmissive;
};

cbuffer cbCameraInfo : register(b1)
{
    matrix      gmtxView : packoffset(c0);
    matrix      gmtxProjection : packoffset(c4);
    float3      gvCameraPosition : packoffset(c8);
};

cbuffer cbGameObjectInfo : register(b2)
{
    matrix      gmtxGameObject : packoffset(c0);
    MATERIAL    gMaterial : packoffset(c4);
    uint        gnTexturesMask : packoffset(c8);
};

cbuffer cbFrameworkInfo : register(b3)
{
    float       gfCurrentTime;
    float       gfElapsedTime;
};

cbuffer cbWaterInfo : register(b4)
{
    matrix      gf4x4TextureAnimation : packoffset(c0);
};

cbuffer cbObjectTextureInfo : register(b6)
{
    matrix      gf4x4TextureSprite : packoffset(c0);
};

```

[셰이더 상수버퍼 관리]

나만의 스프라이트 애니메이션 텍스처 구하기 ->



각 애니메이션 마다 256x256 8x8프레임 텍스처를 구하여 dds파일로 변환하여 적용시켜 보았다.

12.조명

- 조명은 총 10개를 활용하였다. 기본적으로 Directional Light를 적용하였고, 플레이어의 포신에서 빛이 앞으로 나게 하는 SpotLight를 적용하고, Ending Scene을 표현하기 위한 Point Light를 활용하였고, 추가적으로 조명 모드를 POINT_ENEMY_LIGHT=4를 추가하여 적의 위치를 조명을 비추기 위한 조명 모드를 만들었다. 조명은 기본적으로 Directional Light과 Spot Light은 많이 다뤘기 때문에 넘어가고, SPOT Light를 활용을 했는데, 기본적인 조명 연산을 거치게 하는 것이 아닌 PointLight 픽셀 셰이더 함수 내에서 범위 밖의 색상을 특정 색으로 해서 뭔가 자기장 효과를 주는 조명 효과를 나타냈다.

각 적 오브젝트가 살아있다면 해당 PointLight들을 위치를 해당 적 오브젝트로 만들어 플레이어가 식별할 수 있도록 파란색으로 조명효과를 보였다.

```
float4 PointLightToEnemy(int nIndex, float3 vPosition, float3 vNormal, float3 vToCamera)
{
    float3 vToLight = gLights[nIndex].m_vPosition - vPosition;
    float fDistance = length(vToLight);
    if (fDistance <= gLights[nIndex].m_fRange)
    {
        float fSpecularFactor = 0.0f;
        vToLight /= fDistance;
        float fDiffuseFactor = dot(vToLight, vNormal);
        if (fDiffuseFactor > 0.0f)
        {
            if (gMaterial.m_cSpecular.a != 0.0f)
            {
#ifdef _WITHREFLECT
                float3 vReflect = reflect(-vToLight, vNormal);
                fSpecularFactor = pow(max(dot(vReflect, vToCamera), 0.0f), gMaterial.m_cSpecular.a);
#else
#ifdef _WITHLOCAL_VIEWER_HIGHLIGHTING
                float3 vHalf = normalize(vToCamera + vToLight);
#else
                float3 vHalf = float3(0.0f, 1.0f, 0.0f);
#endif
                fSpecularFactor = pow(max(dot(vHalf, vNormal), 0.0f), gMaterial.m_cSpecular.a);
#endif
            }
        }
        float fAttenuationFactor = 1.0f / dot(gLights[nIndex].m_vAttenuation, float3(1.0f, fDistance, fDistance * fDistance));
        return(float4(0.f, 0.f, 0.6f, 0.f));
        //return(((gLights[nIndex].m_cAmbient * gMaterial.m_cAmbient) + (gLights[nIndex].m_cDiffuse * fDiffuseFactor * fAttenuationFactor));
    }
    return(float4(0.f, 0.0f, 0.0f, 0.0f));
}
```

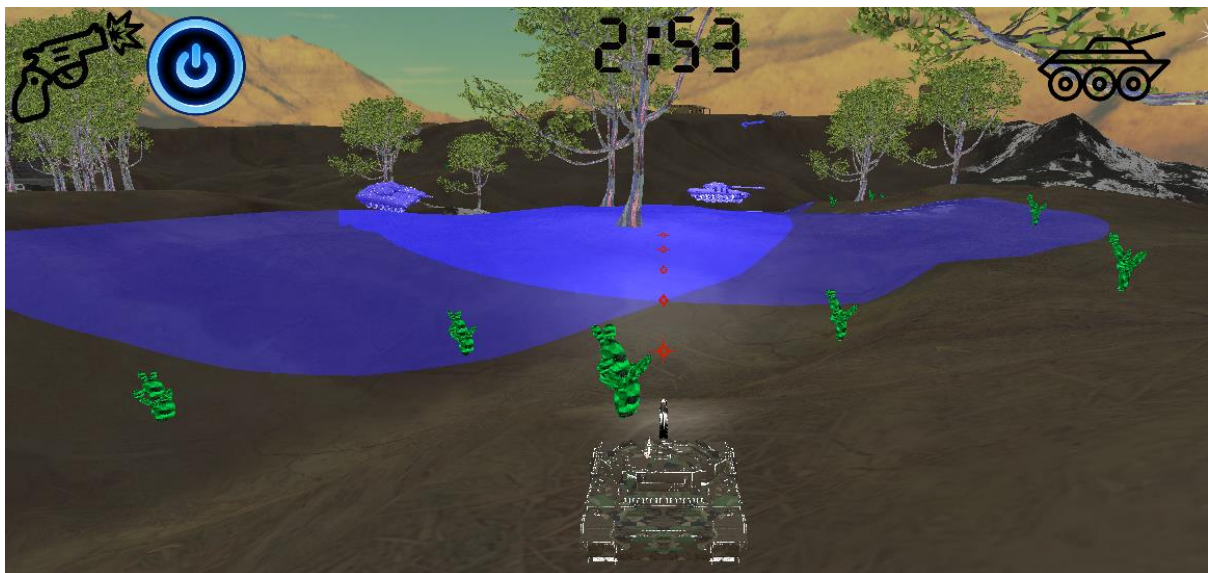
[적 포인트라이트 셰이더 함수]

```

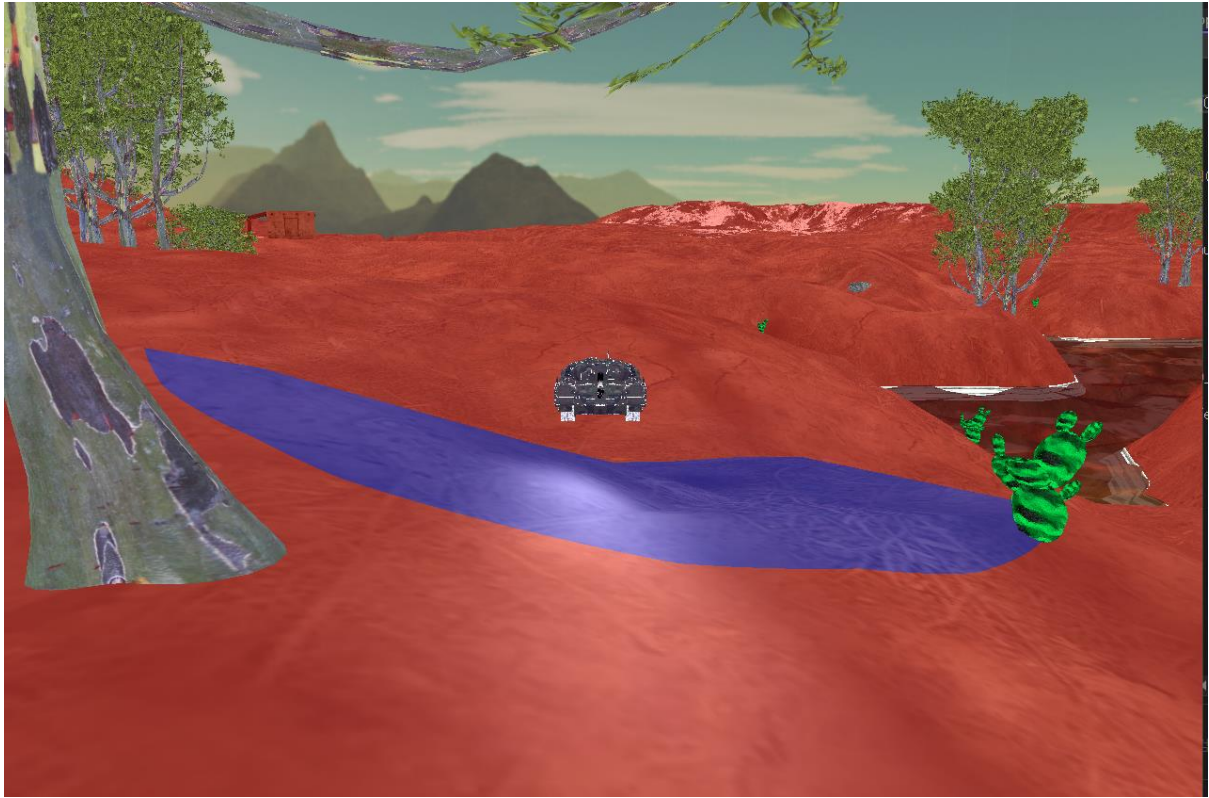
if (gLights[i].m_nType == DIRECTIONAL_LIGHT)
{
    cColor += DirectionalLight(i, vNormal, vToCamera);
}
else if (gLights[i].m_nType == POINT_LIGHT)
{
    cColor += PointLight(i, vPosition, vNormal, vToCamera);
}
else if (gLights[i].m_nType == SPOT_LIGHT)
{
    cColor += SpotLight(i, vPosition, vNormal, vToCamera);
}
else if (gLights[i].m_nType == POINT_ENEMY_LIGHT)
{
    cColor += PointLightToEnemy(i, vPosition, vNormal, vToCamera);
}
}

```

[조명 모드 추가]



[R키를 눌러 적 주위에 파란색으로 영역 표시- 적을 찾기 위해]



[엔딩 씬 중 플레이어 주위에 점점 조명의 범위가 좁혀지는 상황]

13. Enemy Tank Objects

적 탱크는 총 7개로 각각 랜덤한 정규분포된 위치에 위치시켜놓았다. 적 탱크는 플레이어의 탱크와 같은 모델이지만, 텍스처는 모두 다르게 세팅된 탱크이고, 좀더 사막화된 탱크라고 보면 된다. 적 탱크는 기본적으로 미사일을 일정 시간 1.5초 정도마다 쏘지만 따로 충돌처리를 플레이어와 구현하지는 않았다. 기존에 적 탱크 오브젝트의 포신과 터렛의 방향을 플레이어의 위치로부터 방향 벡터를 구해서 자동으로 회전하여 조준하게끔 구현하려 했는데, 실패하였다. 그렇기 때문에 현재는 적 객체를 찾아 처치해야 하는 게임이므로 해당 미사일을 쏘는 것은 그대로 냅두고 적을 찾기 조금이나마 쉽도록 하는 용도로 사용하였다. 적 객체는 랜덤한 위치에서 이동을 하도록 구현되어있으며, 플레이어와 마찬가지로 물에 들어가면 둥둥 떠다니는 애니메이션과 적 총알은 현재 포신이 바라보는 위치로 쏘게 된다. 적 객체도 이동간에 바퀴가 지속적으로 회전하는 상태이며, 플레이어와 마찬가지로 FindFrame으로 각각 바퀴와 포신, 터렛을 객체화 하였다. 적도 미사일을 가지므로 적의 총알 객체도 따로 CTankObjectBullet으로 만들어 FireBullet과 포신 방향에서 쏘는 함수등 플레이와 유사하게 구현하였다. CTankObject인 적 탱크 객체의 Update함수에는 지속적으로 위치를 바꾸면서 일정 시간이 지나면 제자리에서 회전하여 방향을 바꾸게끔 하였고, 지형의 크기인 0~6400은 벗어나지 않도록 하였다. 또한 상시로 총알을 맞았을 때를 조건문으로 검사하여 만약 죽을 시 스프라이트 애니메이션 등 피격 애니메이션 실행을 위해 대기 중이다.

플레이어와 적 탱크 오브젝트의 제일 큰 차이점은 플레이어는 UP벡터가 0,1,0으로 고정되어 있는 상태로 움직인다. 반면에 적 탱크를 유심히 살펴보면 지형의 경사에 따라 기울면서 움직이게 된다. 지형의 굴곡에 따라 Up, Right, Look 벡터 모두 바뀌어 선형적으로 각도를 계산하여 일치시키게끔 구현하여 자연스럽게 움직이는 형태가 된다. 플레이어에 적용시키지 않은 이유는 적을 맞추기 조금 더 까다롭고 카메라의 update함수를 고쳐야하는 문제가 생겨 총알모드가 단발일 때 카메라 update에서 생기는 문제가 있었다.(안 고치면 멀미가 난다)

```

void CTankObject::UpdateTankUpLookRight()
{
    CHeightMapTerrain* pTerrain = (CHeightMapTerrain*)m_pTankObjectUpdatedContext;
    XMFLOAT3 TankObjectPosition = GetPosition();
    XMFLOAT3 TankUp_FLOAT3 = GetUp();
    XMFLOAT3 TankLook_FLOAT3 = GetLook();
    XMFLOAT3 TankRight_FLOAT3 = GetRight();
    XMFLOAT3 TerrainNormal_FLOAT3;
    if (TankObjectPosition.y < 185.f) {
        TerrainNormal_FLOAT3 = XMFLOAT3(0.f, 1.f, 0.f);
    }
    else {
        TerrainNormal_FLOAT3 = pTerrain->GetNormal(TankObjectPosition.x, TankObjectPosition.z);
    }

    XMFLOAT3 UpdateAxis;
    XMVECTOR TankUp_VECTOR = XMLoadFloat3(&TankUp_FLOAT3);
    XMVECTOR TerrainNormal_VECTOR = XMLoadFloat3(&TerrainNormal_FLOAT3);

    XMATRIX RotationMatrix_WORLD = XMMatrixIdentity();

    float Dot = XMVectorGetX(XMVector3Dot(TankUp_VECTOR, TerrainNormal_VECTOR));
    float TotalRotationAngle = acosf(Dot);
    float InterpolationFactor = 0.9f;

    TotalRotationAngle = (1.0f - InterpolationFactor) * TotalRotationAngle;

    if (TotalRotationAngle > 0.0f)
    {
        UpdateAxis = Vector3::CrossProduct(TankUp_FLOAT3, TerrainNormal_FLOAT3, true);
        RotationMatrix_WORLD = XMMatrixRotationAxis(XMLoadFloat3(&UpdateAxis), TotalRotationAngle);
    }

    TankLook_FLOAT3 = Vector3::TransformNormal(TankLook_FLOAT3, RotationMatrix_WORLD);
    TankUp_FLOAT3 = Vector3::TransformNormal(TankUp_FLOAT3, RotationMatrix_WORLD);
    TankRight_FLOAT3 = Vector3::TransformNormal(TankRight_FLOAT3, RotationMatrix_WORLD);

    m_xmf4x4Transform._11 = TankRight_FLOAT3.x; m_xmf4x4Transform._12 = TankRight_FLOAT3.y; m_xmf4x4Transform._13 =
    m_xmf4x4Transform._21 = TankUp_FLOAT3.x; m_xmf4x4Transform._22 = TankUp_FLOAT3.y; m_xmf4x4Transform._23 = TankL

```

[적 탱크 오브젝트 경사로에 따른 Up, Look, Right 벡터 변환]



[미사일을 쏘는 적 탱크]



[경사로에 자기 가는 방향에 맞게 기울어져 가는 모습]



[플레이어의 미사일에 맞고 흔들리면서 3개의 스프라이트 애니메이션 적용& 사운드 적용]



[적 하나씩 처치 후 깎이는 탱크 수]

14. UI

```
#ifdef _WITH_DIRECT2D
void CGameFramework::CreateDirect2DDevice()
{
    UINT nD3D11DeviceFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;
#ifdef _DEBUG || defined(DBG)
    nD3D11DeviceFlags |= D3D11_CREATE_DEVICE_DEBUG;
#endif

    ID3D11Device* pd3d11Device = NULL;
    ID3D12CommandQueue* ppd3dCommandQueues[] = { m_pd3dCommandQueue };
    HRESULT hResult = ::D3D11On12CreateDevice(m_pd3dDevice, nD3D11DeviceFlags, NULL, 0, reinterpret_cast<
    hResult = pd3d11Device->QueryInterface(__uuidof(ID3D11On12Device), (void**)&m_pd3d11On12Device);
    if (pd3d11Device) pd3d11Device->Release();

    D2D1_FACTORY_OPTIONS nD2DFactoryOptions = { D2D1_DEBUG_LEVEL_NONE };
#ifdef _DEBUG || defined(DBG)
    nD2DFactoryOptions.debugLevel = D2D1_DEBUG_LEVEL_INFORMATION;
    ID3D12InfoQueue* pd3dInfoQueue = NULL;
    if (SUCCEEDED(m_pd3dDevice->QueryInterface(IID_PPV_ARGS(&pd3dInfoQueue))))
    {
        D3D12_MESSAGE_SEVERITY pd3dSeverities[] =
        {
            D3D12_MESSAGE_SEVERITY_INFO,
        };

        D3D12_MESSAGE_ID pd3dDenyIds[] =
        {
            D3D12_MESSAGE_ID_INVALID_DESCRIPTOR_HANDLE,
        };

        D3D12_INFO_QUEUE_FILTER d3dInforQueueFilter = { };
        d3dInforQueueFilter.DenyList.NumSeverities = _countof(pd3dSeverities);
        d3dInforQueueFilter.DenyList.pSeverityList = pd3dSeverities;
        d3dInforQueueFilter.DenyList.NumIDs = _countof(pd3dDenyIds);
        d3dInforQueueFilter.DenyList.pIDList = pd3dDenyIds;

        pd3dInfoQueue->PushStorageFilter(&d3dInforQueueFilter);
    }
    pd3dInfoQueue->Release();
#endif

    hResult = ::D2D1CreateFactory(D2D1_FACTORY_TYPE_SINGLE_THREADED, __uuidof(ID2D1Factory3), &nD2DFacto
```

UI는 DirectX11의 장치를 이용하여 2D이미지를 처리하는 가우시안 블러, 비트맵 소스, 엣지 디텍션 등을 배열 형태로 가지고 있어 각 png나 이미지를 로드하여 UI형태로 띄웠다. UI는 시작 타이틀, 시작 스페이스 이미지, 연발/단발 모드 UI, R조명 UI, 시간 UI, 점수 UI, 엔딩 씬 관련 UI 등 여러가지를 넣었다. 그 중 시간과 같은 경우에는 처음에 그냥 스

탑 와치 형태로 만들었는데, 시간 제한을 두고 카운트 다운을 구현하기 위해서 디지털 번호를 투명한 배경에서 그리고 있는 이미지를 구해야 했다. 자료가 별로 없어 해맸지만 그나마 쓸만한 사진을 찾아 활용하였다. 이렇게 디지털 숫자가 0~9까지 있는 사진에서 각 영역을 계산하여 UI를 초단위마다 출력 범위를 다르게 해주어야 한다.

```
if (elapsedTimeInSeconds >= 1.0f) {  
    if (ones_x == 0.f && ones_y == 0.f && tens_x == 0.f && tens_y == 0.f) {  
        if (mins_ones_x > 0.f) {  
            mins_ones_x -= 900.f;  
        }  
        tens_y = 1400.f;  
        ones_x = 3600.f;  
        ones_y = 1400.f;  
    }  
    else {  
        ones_x -= 920;  
        ones_cnt++;  
    }  
    if (ones_cnt == 5) {  
        /* m_pScene->Lose = true;  
        m_pScene->End_Game = true; */  
        ones_y -= 1400.f;  
        ones_x = 3600.f;  
    }  
    else if (ones_cnt == 10) {  
        ones_cnt = 0;  
        ones_x = 3600.f;  
        ones_y = 1400.f;  
        tens_cnt++;  
        if (tens_cnt == 1) {  
            m_pScene->Lose = true;  
            m_pScene->End_Game = true;  
            tens_y = 0.f;  
            tens_x = 3600.f;  
        }  
        else {  
            tens_x -= 900.f;  
        }  
        if (tens_cnt == 6) {  
            tens_x = 0;  
            tens_y = 1400.f;  
        }  
        mins_ones_cnt++;
```

[시간 1초 지날때마다 영역 바꾸는]



[시간 출력을 위한 png]

점수 같은 경우에는 각 적 탱크 오브젝트들이 죽은 개수만큼 띄워야하므로 값을 계산해서 마찬가지로 해당 이미지에서 영역을 계산하여 출력하였다.

```
if (m_pScene->scene_Mode == SceneMode::Playing) {  
    switch (m_pScene->n_deadTank) {  
        case 1:  
            Score_x = 150.f;  
            break;  
        case 2:  
            Score_x = 0.f;  
            break;  
        case 3:  
            Score_x = 450.f;  
            Score_y = 0.f;  
            break;  
        case 4:  
            Score_x = 300.f;  
            break;  
        case 5:  
            Score_x = 150.f;  
            break;  
        case 6:  
            Score_x = 0.f;  
            break;  
        case 7:  
            m_pScene->Win = true;  
            m_pScene->End_Game = true;  
            break;  
    }  
    D2D_POINT_2F d2dPointScoreUI = { 2300, 0 };  
    D2D_RECT_F d2dRectScoreUI = { Score_x, Score_y, Score_x+150.f, Score_y+213.f }; // UI
```

[탱크 죽은 개수에 맞춰 영역 출력]



[시작 화면 UI]



[인 게임 UI]



[종료 씬 UI]

15. Descriptor Heap & Root Parameter

기존 코드를 지속적으로 사용함에 따라 오브젝트들을 로드하면서 생기는 오류들이 많았다. 일단 프레임 레이트가 디버그 모드로 돌렸을 때 매우 느린 현상이 나타났다. 이에 추측되는 점은 셰이더 내부에서 디스크립터 힙을 관리하고, 각 셰이더의 BuildObjects를 호출할 때 마다 새로운 디스크립터 힙을 만들어 Set하는 과정이 비용소모가 크다는 것이었다. 그러므로 씬으로 디스크립터 힙을 옮기고 하나의 클래스에서 관리하도록 하였다. 그렇게 한 결과 프레임 레이트도 올라가고 DrawIndexed 경고도 말끔히 지워졌다.

[illegible]

-> 이전 오류들

```

m_pDescriptorHeap = new CDescriptorHeap();
CreateObvSrvDescriptorHeaps(pDevice, 0, 4+9+3+5+2+4+5 +3+5+3+1+30); //총알(4) 건물들3가지(9) 윈드밀(3) 탱크(5), 돌식물(2) 나무(4) 풀
//물(3) 지형 (5) 스포라이트 3 스카이라이프 1

m_nDotBillboard = 8;
m_ppDotBillboard = new CBillboardObject * [m_nDotBillboard];

```

추가적으로 모델을 로드하면서 텍스처를 생성할 때 각종 텍스처의 유무에 따라 루트파라미터 인덱스를 지정하여 뷰를 만들어주었다. 하지만 루트파라미터를 Range하나하나 할당하여 세팅되어있다보니 루트 파라미터 낭비가 너무 심하였다. 그렇기 때문에 하나의 Range를 사용하여 텍스처 배열로 반도록 바꾸고 나머지 파라미터는 두가지의 경우에 #define하나만 지우고 생성함에따라 유기적으로 바뀌게 모두 파라미터 인덱스를 정의해주었다.

또한 이번에는 루트 파라미터를 무려 17개나 사용하고, Range는 10개나 사용하였다. 다음에는 이를 보완하기 위해 같이 묶을 수 있는 것은 묶어야 할 것 같다. 사실 이번에도 최대한 줄일 수 있는 건 줄였지만, 가장 큰 오점은 아무래도 SRV를 단일로 받는 파라미터가 많아 낭비가 심했다는 점이 있었다. 만약 다음이나 나중의 프로젝트에서 더 많은

구현요소가 생긴다면 루트 파라미터가 여유가 있어야 하는데, 이 경우에는 현재 여유가 없어 문제요소 중 하나이다.

```
//#define _WITH_STANDARD_TEXTURE_MULTIPLE_DESCRIPTOR

#define PARAMETER_STANDARD_TEXTURE 3 //카메라 , 월드 , 조명은 같음
#ifdef _WITH_STANDARD_TEXTURE_MULTIPLE_DESCRIPTOR

#define PARAMETER_SKYBOX_CUBE_TEXTURE 10
#define PARAMETER_TERRAIN_TEXTURES 11
#define PARAMETER_WATER_TEXTURES 12
#define PARAMETER_2D_TEXTURE 13

#define PARAMETER_TIME_CONSTANTS 14
#define PARAMETER_WATER_ANIMATION_MATRIX 15
#define PARAMETER_SPRITE_ANIMATION_MATRIX 16

#else

#define PARAMETER_SKYBOX_CUBE_TEXTURE 4
#define PARAMETER_TERRAIN_TEXTURES 5
#define PARAMETER_WATER_TEXTURES 6
#define PARAMETER_2D_TEXTURE 7

#define PARAMETER_TIME_CONSTANTS 8
#define PARAMETER_WATER_ANIMATION_MATRIX 9
#define PARAMETER_SPRITE_ANIMATION_MATRIX 10
```

루트 시그니처는 결국 #define WithStandardTextureMultipleDescriptor의 유무에 따라 다른 구조의 루트시그니처가 만들어지도록 구현하였다. 루트 시그니처가 너무 길어 코드를 참조해보면 알 수 있다.

16. Sound

사운드를 넣는 것은 거의 마지막에 해서 시간 관계상 적을 폭격했을 때만 사운드를 입히는 식으로 연습해보았다. 사운드를 넣기 위해서는 FMOD::에 있는 System과 Sound Channel등의 객체를 특정 라이브러리에서 불러와 사용하였다. 게임에서 많이 쓰는 기법이라 먼저 연습차원에서 적용해보았다.

```
GameSound::GameSound()
{
    result = FMOD::System_Create(&soundSystem);

    result = soundSystem->init(32, FMOD_INIT_NORMAL, extradriverdata); //
    result = soundSystem->createSound("Sound/TankExplosion.wav", FMOD_DEFAULT, 0, &BackGroundSound);
    result = BackGroundSound->setMode(FMOD_LOOP_NORMAL); //LOOP_NORMAL ->무한 루프 FMOD_LOOP_OFF->한번?

    result = soundSystem->playSound(BackGroundSound, 0, true, &BackGroundChannel); //true가 정지.
    BackGroundChannel->setVolume(0.5f);
}
```

사운드를 적용할 때 많이 사용하는 playSound함수를 사용해서 루프를 무한 루프로 실행할지 안할지 여부를 정하여 적이 피격 당했을 때 실행하게끔 하였다. 하지만 루프를 돌다보니 On/Off를 하고 다시 키거나 하면 루프를 돌던 브금이 그 시점부터 다시 이어져 원활한 폭격 사운드가 매번 깔끔하게 실행되지 않는 문제가 생겼다. 이를 해결하기 위해 직접 wav파일을 수정하여 폭격음을 2.5초에 맞추어 짧랐다. 2초동안의 사운드가 발생하므로 적이 피격당했을 때 사운드를 2.5초동안 키면 다음 루프때 처음부터 사운드가 발생하는 식으로 원활히 사운드를 활용할 수 있었다.

```
if (Game_Sound.Sound_ON) {
    if (Game_Sound.SoundElapsedTime < 2.5f) {
        Game_Sound.SoundElapsedTime += fTimeElapsed;
        Game_Sound.PlayOpeningSound();
    }
    else {
        Game_Sound.PauseOpeningSound();
        Game_Sound.Sound_ON = false;
        Game_Sound.SoundElapsedTime = 0.f;
    }
}
```

[2.5초동안 실행하는]